Giovanni Alvarenga Vieira Soares

Transformers na Predição de Ataques de Negação de Serviço Distribuído

Belo Horizonte, Minas Gerais
2025

Giovanni Alvarenga Vieira Soares

Transformers na Predição de Ataques de Negação de Serviço Distribuído

Relatório apresentado como requisito da disciplina de Monografia em Engenharia de Sistemas do Curso de Bacharelado em Engenharia de Sistemas da UFMG

Universidade Federal de Minas Gerais
Instituto de Engenharia
Departamento de Engenharia Elétrica

Orientador: Prof.ª Dra. Michele Nogueira Lima
Coorientadores: Dr. Anderson Bergamini de Neira
Mestre Ligia F. Borges

Belo Horizonte, Minas Gerais
2025

Resumo

Os Ataques de Negação de Serviço Distribuído (em inglês, *Distributed Denial of Service* – DDoS) volumétricos são uma classe de ataques cibernéticos que visam negar o acesso de usuários legítimos a serviços ou recursos compartilhados por meio de uma inundação de requisições ao alvo escolhido, utilizando uma rede de dispositivos infectados, conhecida como *botnet*. Embora existam diversas contramedidas para se defender dessa ameaça, muitas vezes as vítimas não conseguem reagir a tempo devido à súbita iminência dos ataques. Em resposta a esse desafio, pesquisadores têm desenvolvido soluções para prever ataques DDoS, analisando na rede sinais deixados pelos cibercriminosos antes do ataque. Em resposta a esse desafio, o aprendizado profundo (em inglês, *Deep Learning*) tem se mostrado uma abordagem promissora para a detecção e previsão de ataques DDoS. Este trabalho de conclusão de curso realiza uma avaliação da aplicação de um modelo de Transformers, investigando o *Feature Tokenizer Transformer*(*FT-Transformer*) como uma solução para prever esses ataques. Esta avaliação é inovadora, ao explorar sua capacidade de generalização entre diferentes conjuntos de dados e utiliza uma análise dos *logits* gerados pelo modelo para compreender melhor suas decisões e identificar padrões de confiança. Após uma análise dos principais modelos disponíveis, o *FT-Transformer* foi escolhido devido à sua habilidade de processar dados contínuos por meio de incorporações categóricas e pela sua eficiência em capturar dependências de longo alcance. Os resultados deste estudo mostram que o FT-Transformer é capaz de lidar com o desbalanceamento de classe do tráfego de rede e generalizar, com uma acurácia de 95%, entre diferentes *datasets*, o que pode contribuir para o desenvolvimento de sistemas de segurança mais robustos, permitindo às organizações implementar medidas preventivas e mitigar os impactos de ataques DDoS de maneira eficaz.

Palavras-chaves: DDoS; Transformers; Predição; Aprendizado Profundo; FT-Transformer.

Abstract

Volumetric Distributed Denial of Service (DDoS) attacks are a class of cyberattacks aimed at denying legitimate users access to shared services or resources by flooding the chosen target with a massive number of requests, using a network of infected devices known as a botnet. Although various countermeasures exist to defend against this threat, victims often fail to respond in time due to the sudden imminence of such attacks. In response to this challenge, researchers have developed solutions to predict DDoS attacks by analyzing signals left by cybercriminals on the network prior to the attack. Deep Learning has emerged as a promising approach for detecting and predicting DDoS attacks. This thesis evaluates the application of a Transformer-based model, investigating the *Feature Tokenizer Transformer* (*FT-Transformer*) as a solution for predicting these attacks. This evaluation is innovative in exploring its ability to generalize across different datasets and employing an analysis of the model's *logits* to better understand its decisions and identify confidence patterns. After analyzing the main models available, the *FT-Transformer* was chosen for its ability to process continuous data through categorical embeddings and its efficiency in capturing long-range dependencies. The results of this study demonstrate that the FT-Transformer can handle the class imbalance in network traffic and generalize with 95% accuracy across different datasets. This capability can contribute to the development of more robust security systems, enabling organizations to implement preventive measures and effectively mitigate the impacts of DDoS attacks.

Key-words: DDoS; Transformers; Prediction; Deep Learning; FT-Transfoormer

Lista de ilustrações

Figura 1 – Ilustra a arquitetura do *Transformer* original, que consiste em um codificador e um decodificador, cada um formado por várias camadas idênticas. O codificador processa a entrada através de camadas de *Multi-Head Attention* e *Feed-Foward*. O decodificador segue um processo similar, mas inclui uma atenção cruzada para integrar as informações do codificador. As conexões residuais e a normalização de camada são aplicadas em cada bloco. Figura baseada em (Vaswani et al., 2017) . . 19

Figura 2 – Exemplo do processo de transformação da frase “Oi como vai você” em *embeddings* de entrada. Cada palavra da frase é dividida em vetores de tamanho 3, com cada posição do vetor contendo um número que representa semanticamente o *token*. No caso da palavra “Oi”, por exemplo, ela é convertida em um vetor de tamanho 3, onde cada valor numérico é uma representação densa da informação. Esse processo transforma as palavras em um formato numérico contínuo, permitindo que o modelo capte relações semânticas e sintáticas entre os *tokens*. Figura adaptada de um conjunto de outras imagens da literatura associado ao modelo (Gorishniy et al., 2023). 21

Figura 3 – Mostra uma matriz 4x4 que ilustra como o mecanismo de atenção calcula a similaridade entre as palavras da frase "Oi como vai você". Cada célula da matriz contém um valor que representa o grau de atenção entre os tokens (palavras no caso). Por exemplo, o valor 0.7 na posição (1,1) indica que a palavra "Oi"tem forte atenção sobre si mesma, enquanto valores menores, como 0.1, indicam menor atenção entre palavras diferentes. Figura adaptada de um conjunto de outras imagens da literatura associado ao modelo (Gorishniy et al., 2023). . . 23

Figura 4 – Exemplo do mecanismo *look-ahead mask*, onde os valores de atenção para posições futuras são mascarados, assegurando que as previsões dependam apenas dos *tokens* já processados. Figura adaptada de um conjunto de outras imagens da literatura associado ao modelo (Gorishniy et al., 2023). 25

Figura 5 – Fluxo da saída no Transformer: a transformação dos vetores gerados pelo decodificador em probabilidades, utilizando o classificador linear seguido pela função *softmax*. Figura adaptada de um conjunto de outras imagens da literatura associado ao modelo (Gorishniy et al., 2023). . . 27

Figura 6 – A arquitetura do *FT-Transformer*. Em primeiro lugar, o *Feature Tokenizer* transforma dados em *emdeddings*. Estes são então processados pelo módulo *Transformer* e a representação final do *token* [CLS] é usada para previsão. Figura baseada em (Gorishniy et al., 2023) 28
Figura 7 – Processo do *Feature Tokenizer*, ilustrando a transformação de duas características categóricas e três características numéricas em representações vetoriais uniformes. Essas representações são concatenadas e recebem a adição do *token* [CLS] para serem processadas pelo modelo. Baseado em (Gorishniy et al., 2023). 29
Figura 8 – Arquitetura do *TabTransformer*, que utiliza mecanismos de atenção para processar interações entre características categóricas, enquanto os atributos numéricos são processados separadamente fora do mecanismo de atenção. Baseado em (Huang et al., 2020). 30
Figura 9 – Gráfico comparando os valores preditos (*logits*) com os valores originais no Conjunto 1 - Cenário 1. 38
Figura 10 – Gráfico comparando os valores preditos (*logits*) com os valores originais no Conjunto 1 - Cenário 2. 39
Figura 11 – Gráfico comparando os valores preditos (*logits*) com os valores originais no Conjunto 1 - Cenário 3. 40
Figura 12 – Gráfico comparando os valores preditos (*logits*) com os valores originais no Conjunto 2 - Cenário 1. 41
Figura 13 – Gráfico comparando os valores preditos (*logits*) com os valores originais no Conjunto 2 - Cenário 2. 42
Figura 14 – Gráfico comparando os valores preditos (*logits*) com os valores originais no Conjunto 2 - Cenário 3. 43
Figura 15 – Gráfico comparando os valores preditos (*logits*) com os valores originais no Conjunto 2 - Cenário 4. 44
Figura 16 – Gráfico comparando os valores preditos (*logits*) com os valores originais no Conjunto 2 - Cenário 5. 45
Figura 17 – Gráfico comparando os valores preditos (*logits*) com os valores originais no Conjunto 2 - Cenário 6. 45
Figura 18 – Gráfico comparando os valores preditos (*logits*) com os valores originais no Conjunto 3 - Cenário 1. 47
Figura 19 – Gráfico comparando os valores preditos (*logits*) com os valores originais no Conjunto 3 - Cenário 2. 48
Figura 20 – Gráfico comparando os valores preditos (*logits*) com os valores originais no Conjunto 4. 49
Figura 21 – Gráfico comparando os valores preditos (*logits*) com os valores originais no Conjunto 5. 51

Lista de tabelas

Tabela 1 – Atributos iniciais dos conjuntos de dados. Tabela baseada em (Neira; Borges et al., 2023). 33
Tabela 2 – Matriz de confusão Conjunto 1 - Cenário 1. 38
Tabela 3 – Matriz de confusão Conjunto 1 - Cenário 2. 39
Tabela 4 – Matriz de confusão Conjunto 1 - Cenário 3. 40
Tabela 5 – Matriz de confusão Conjunto 2 - Cenário 1. 41
Tabela 6 – Matriz de confusão Conjunto 2 - Cenário 2. 42
Tabela 7 – Matriz de confusão Conjunto 2 - Cenário 3. 43
Tabela 8 – Matriz de confusão Conjunto 2 - Cenário 4. 44
Tabela 9 – Matriz de confusão Conjunto 3 - Cenário 1. 46
Tabela 10 – Matriz de confusão Conjunto 3 - Cenário 2. 48
Tabela 11 – Matriz de confusão Conjunto 4 49
Tabela 12 – Matriz de confusão Conjunto 5 50

Lista de abreviaturas e siglas

ACS	*Adaptive Conditional Score*
ARIMA	*Autoregressive Integrated Moving Average*
CLS	*Classification*
CNN	*Convolutional Neural Networks*
DDoS	*Distributed Denial of Service*
FFNN	Feed-Foward *Neural Network*
FN	Falso Negativo
FP	Falso Positivo
FT-Transformer	*Feature Tokenizer Transformer*
IoT	*Internet of Things*
K	*Key*
MLP	*Multi-layer Perceptron*
NLP	*Natural Language Processing*
Q	*Query*
ReLU	*Rectified Linear Unit*
RNN	*Recurrent Neural Networks*
TabTransformer	*Tabular Transformer*
V	*Value*
VN	Verdadeiro Negativo
VP	Verdadeiro Positivo

Sumário

1 INTRODUÇÃO 10
1.1 Objetivo Geral 11
1.2 Objetivos Específicos 12
1.3 Contribuições deste Trabalho 12
1.4 Organização do Texto 13

2 FUNDAMENTOS E TRABALHOS RELACIONADOS 15
2.1 Ataque de Negação de Serviço Distribuído 15
2.1.1 Contramedidas aos ataques de DDoS 16
2.1.1.1 Prevenção 16
2.1.1.2 Detecção 16
2.1.1.3 Reação 17
2.1.1.4 Predição 18
2.2 Transformers 18
2.2.1 Codificador 19
2.2.1.1 Input Embedding 20
2.2.1.2 Positional Encoding 20
2.2.1.3 Mecanismo de Atenção 21
2.2.1.4 *Feed-Foward Neural Network* (FFNN) 23
2.2.2 Decodificador 24
2.2.3 Saída 26
2.3 FT-Transformer 26
2.3.1 Feature Tokenizer 28
2.3.1.1 Token CLS 29
2.4 Trabalhos correlacionados 29

3 METODOLOGIA 32
3.1 Conjunto de Dados 32
3.2 Métricas para avaliação 35
3.3 Transformação e preparação dos dados 35

4 RESULTADOS 37
4.1 Experimentos com conjunto de dados 1 37
4.1.1 Cenário de treinamento 1 38
4.1.2 Cenário de treinamento 2 39
4.1.3 Cenário de treinamento 3 40

4.2 Experimentos com conjunto de dados 2 . . . 40
4.2.1 Cenário de treinamento 1 . . . 41
4.2.2 Cenário de treinamento 2 . . . 42
4.2.3 Cenário de treinamento 3 . . . 43
4.2.4 Cenário de treinamento 4 . . . 44
4.3 Experimentos com conjunto de dados 3 . . . 46
4.3.0.1 Cenário de treinamento 1 . . . 46
4.3.1 Cenário de treinamento 2 . . . 48
4.4 Experimentos misturando o conjunto de dados 1 e 2 . . . 48
4.5 Experimentos misturando o conjunto de dados 1 e 3 . . . 50
4.6 Discussão . . . 52

5 CONCLUSÃO . . . 54
5.1 Recomendações Futuras . . . 55

Referências . . . 56

1 Introdução

Os ataques de negação de serviço distribuído (em inglês, *Distributed Denial of Service* – DDoS) representam uma das mais graves ameaças à segurança cibernética atual (Lopez; Buscetto, 2021). Em geral, uma categoria desses ataques denominada de DDoS volumétrico se caracteriza por inundar servidores, serviços ou redes com um tráfego massivo, prejudicando o acesso de usuários legítimos e causando interrupções significativas. Estes ataques têm se tornado cada vez mais frequentes e sofisticados (Cisco, 2023), o que ressalta a necessidade de desenvolver mecanismos de defesa avançados para lidar com os ataques. Em 2024, houve um aumento significativo nos ataques de DDoS. No primeiro trimestre do ano, as defesas automatizadas da Cloudflare mitigaram 4,5 milhões de ataques DDoS, representando um aumento de 50% em relação ao período homólogo (Tremante; Zejnilovic; Newcomb, 2024). Alguns desses ataques atingiram taxas de até 2 terabits por segundo, com um dos maiores incidentes envolvendo um provedor de hospedagem asiático. Um exemplo notável de ataque DDoS em 2024 foi dirigido contra serviços estatais franceses em março (Giannotti, 2024). Este ataque, descrito como de "intensidade sem precedentes" pelo escritório do Primeiro-Ministro francês, afetou mais de 300 domínios web e 177.000 endereços IP associados ao governo, causando sérias interrupções em sites de serviços públicos importantes.

A detecção e predição de ataques DDoS são cruciais para mitigar seus impactos antes que causem danos significativos. Tradicionalmente, métodos de detecção de anomalias, como listas negras e heurísticas baseadas em assinaturas, têm sido usados. No entanto, essas abordagens enfrentam limitações, como a incapacidade de identificar rapidamente novas formas de ataques e a dificuldade de analisar grandes volumes de dados em tempo real (Jain; Shukla; Goel, 2024). A crescente sofisticação dos ataques DDoS, com técnicas como a variação dinâmica de padrões de tráfego e a utilização de *botnets*, torna a detecção ainda mais desafiadora. Nesse contexto, modelos de aprendizado de máquina emergem como uma alternativa promissora, oferecendo potencial para melhorar a precisão e a eficiência na detecção de padrões anômalos (Dehkordi; Soltanaghaei; Boroujeni, 2021).

Entre esses modelos, os modelos Transformers, ou apenas Transformers neste trabalho, destacam-se por sua eficácia em capturar padrões complexos em dados sequenciais. O modelo original de Transformers, proposto por (Vaswani et al., 2017) no trabalho *Attention Is All You Need*, revolucionou o processamento de linguagem natural ao introduzir mecanismos de atenção que permitem a captura de dependências de longo alcance em dados sequenciais. Predição, no escopo deste trabalho, refere-se à capacidade de prever a ocorrência de ataques DDoS antes que a sobrecarga de tráfego malicioso inicie, permitindo a implementação de medidas preventivas.

As pesquisas de (Huang et al., 2020) e (Vyas, 2024) com o modelo *TabTransformer*

evidenciaram como o uso de *embeddings* contextuais melhora o desempenho em tarefas de classificação e regressão em dados tabulares. Esses mecanismos de atenção são capazes de identificar padrões sutis e interdependentes, uma característica essencial para a predição de ataques DDoS, onde os indícios de comportamento malicioso estão distribuídos ao longo do tempo. Essas contribuições da literatura indicam que os modelos de Transformers têm potencial não só para o Processamento de Linguagem Natural (do inglês, *Natural Language Processing* - NLP), mas também para diversas outras aplicações, incluindo a predição de ataques DDoS. Além disso, o estudo "Revisiting Deep Learning Models for Tabular Data" de (Gorishniy et al., 2023) apresentou o *FT-Transformer*.

No TCC1, foi investigada a aplicação do FT-Transformer na predição de ataques DDoS, utilizando um conjunto de dados limitado para avaliar sua viabilidade. Os resultados obtidos indicaram o potencial do modelo, mas também destacaram a necessidade de trabalhar com bases de dados mais robustos e representativos para uma avaliação mais precisa. Neste TCC2 deu-se continuidade ao estudo explorando a aplicação do FT-Transformer em cenários mais diversificados e avaliando sua eficácia com dados tabulares mais robustos e abrangentes. Avalia-se a implementação e validação do modelo em novos conjuntos de dados voltados para DDoS. Para isso, foram utilizados os *datasets* CTU-13 (García et al., 2014) e CICDDoS2019 (Sharafaldin et al., 2019), que apresentam diferentes características e complexidades relacionadas ao tráfego de rede. Esses experimentos permitem prever a probabilidade de ocorrência de ataques em diferentes cenários e contribuem para a construção de sistemas de segurança mais eficientes.

Este trabalho ampliou o uso de Transformers para dados tabulares, demonstrando sua eficácia na modelagem de dados não estruturados. A pesquisa destacou que essa arquitetura supera significativamente métodos tradicionais, como árvores de decisão e redes neurais profundas, em termos de precisão e robustez, consolidando-se como uma abordagem eficaz para modelagem de dados não estruturados. Essas conclusões foram alcançadas a partir dos resultados obtidos nos experimentos, que evidenciaram a capacidade do *FT-Transformer* de capturar relações complexas entre atributos e identificar padrões sutis nos dados. Observou-se um desempenho consistente do modelo, com altos valores de precisão e acurácia, principalmente em cenários mais equilibrados, além de uma maior confiança nas predições, conforme indicado pela análise dos *logits*. Além disso, a flexibilidade do *FT-Transformer* em se adaptar a diferentes configurações e ajustes de hiperparâmetros demonstrou sua robustez e potencial para aprimoramento contínuo, reforçando sua viabilidade como uma solução eficaz para predição de ataques DDoS.

1.1 Objetivo Geral

Este Trabalho de Conclusão de Curso 2 visa aprofundar a investigação sobre a eficácia dos modelos de Transformers, com ênfase no *FT-Transformer* (Gorishniy et al., 2023),

para a predição de ataques DDoS. Essa etapa dá continuidade à análise realizada no TCC1, onde o potencial do *FT-Transformer* foi identificado, mas também foram evidenciadas limitações importantes. Entre essas limitações, destacou-se a necessidade de *datasets* mais robustos e diversificados para uma avaliação mais precisa e confiável do modelo.

Nesta etapa, busca-se validar a capacidade do modelo de detectar padrões anômalos em cenários mais próximos da realidade. Para isso, o seu desempenho foi avaliando em condições variadas e otimizando seus parâmetros para melhorar a precisão e a eficiência. Os resultados demonstrados incluem a identificação de limitações e vantagens do *FT-Transformer*, espera-se que eles contribuam para a implementação de sistemas de segurança mais robustos, apoiando o aumento da resiliência das redes contra ataques DDoS (Tremante; Zejnilovic; Newcomb, 2024) e proporcionando às organizações a capacidade de reagir de maneira proativa e eficiente diante deste tipo de ameaça.

1.2 Objetivos Específicos

O objetivo da primeira fase do trabalho, Monografia em Engenharia de Sistemas I, foi investigar e implementar o uso de modelos baseados em Transformers para a predição de ataques de negação de serviço distribuído (DDoS) em sistemas, utilizando dados tabulares como entrada. Buscou-se explorar a aplicação de Transformers, originalmente desenvolvidos para processamento de linguagem natural (Natural Language Processing – NLP), na análise de padrões temporais e detecção de anomalias em tráfego de rede. O trabalho envolveu a utilização de dados de estudos anteriores (Lima et al., 2023) para preparar um conjunto de dados tabulares representativos de atividades de rede, usados para treinar, testar e validar o modelo. Além disso, foi realizada a adaptação de arquiteturas de Transformers, como o *FT-Transformer*, para aprender e prever padrões relacionados a ataques DDoS. Os experimentos iniciais revelaram que, embora o modelo tenha mostrado potencial, a quantidade limitada de dados comprometeu a eficácia dos resultados.

A segunda fase do trabalho, Monografia em Engenharia de Sistemas II, foca na superação dessa limitação. Nesta etapa, o objetivo é avaliar o desempenho do *FT-Transformer* utilizando bases de dados mais robustos e diversificados, permitindo uma análise mais aprofundada e realista. Serão realizados ajustes nos parâmetros do modelo, testes com abordagens adicionais de pré-processamento de dados. Espera-se, ao final dessa fase, confirmar a viabilidade do *FT-Transformer* para prever ataques DDoS eficazmente e contribuir para o avanço de sistemas de segurança baseados em aprendizado profundo.

1.3 Contribuições deste Trabalho

Este Trabalho de Conclusão de Curso 2 representou um avanço significativo em relação ao estudo inicial conduzido no TCC1, aprofundando a aplicação do *FT-Transformer*

na predição de ataques de negação de serviço distribuído (DDoS). Uma das principais contribuições foi a validação do modelo em novos cenários experimentais utilizando conjuntos de dados mais robustos e variados, com destaque para a análise dos *logits* gerados, que permitiram compreender melhor as decisões do modelo e avaliar sua confiança nas predições. Os experimentos realizados demonstraram a capacidade do *FT-Transformer* de alcançar resultados notáveis, com acurácia média de 95% e precisão de até 93% na identificação de fluxos maliciosos. Além disso, os valores dos *logits* indicaram maior confiança nas predições de ataques, evidenciando o potencial do modelo para lidar com conjuntos de dados mais representativos e variados.

Os resultados obtidos mostraram que o modelo apresentou precisão de 99% para fluxos normais em todos os experimentos e conseguiu superar limitações identificadas anteriormente no TCC1, como a baixa confiança nas predições de fluxos maliciosos. Além disso, foram identificadas outras melhorias significativas, incluindo a generalização entre diferentes conjuntos de dados, que foi validada por meio dos *logits* gerados pelo modelo, indicando maior certeza nas decisões preditivas. No experimento final, que envolveu treinamento com a captura 51 do CTU-13 (García et al., 2014) e teste com o CICDDoS2019 (Sharafaldin et al., 2019), o *FT-Transformer* alcançou F1-scores superiores a 94% para ambas as classes, destacando sua robustez em cenários distintos. Adicionalmente, o trabalho contribuiu com uma análise detalhada do impacto do balanceamento dos dados e das divisões de treinamento, validação e teste no desempenho do modelo. Foi explorada a utilização de técnicas como aumento da complexidade arquitetural e ajuste de hiperparâmetros, o que melhorou o entendimento sobre os limites e potenciais da aplicação de Transformers em segurança cibernética.

Essas contribuições não apenas validam a eficácia do *FT-Transformer* para a predição de ataques DDoS, mas também oferecem um alicerce sólido para estudos futuros. Esses estudos podem explorar novos cenários de aplicação e ampliar a diversidade de dados utilizados no treinamento, garantindo maior generalização do modelo. Ao longo deste TCC2, os resultados reforçaram a importância de estratégias avançadas de aprendizado profundo no combate a ameaças cibernéticas crescentes, consolidando o papel do *FT-Transformer* como uma solução promissora para a segurança de redes.

1.4 Organização do Texto

O restante deste trabalho está organizado como segue. No Capítulo 2, é apresentada uma revisão da literatura, abordando os principais conceitos e estudos relacionados à predição de ataques DDoS e modelo de Transformer. O Capítulo 3 descreve a metodologia utilizada para a coleta e preparação dos dados, além de apresentar a metodologia de avaliação. O Capítulo 4 detalha a metodologia utilizada para a coleta e preparação dos dados, além de apresentar a metodologia de avaliação e os resultados dos experimentos e a

análise comparativa dos modelos testados. Finalmente, no Capítulo 5, são discutidas as conclusões do estudo, bem como sugestões para trabalhos futuros e possíveis melhorias na detecção e predição de ataques DDoS.

2 Fundamentos e Trabalhos Relacionados

Neste capítulo, são apresentados os fundamentos teóricos e os trabalhos relacionados que embasam este estudo. Primeiramente, discute-se a problemática dos ataques de negação de serviço distribuído (em inglês, *Distributed Denial of Service* – DDoS), com ênfase na subcategoria volumétrica, destacando sua relevância no cenário atual de segurança cibernética e as principais contramedidas existentes. Em seguida, aborda-se a evolução dos modelos de aprendizado profundo, culminando na introdução dos Transformers, que revolucionaram o processamento de dados sequenciais, como texto e, mais recentemente, dados tabulares. Por fim, apresenta-se o modelo *FT-Transformer*, que foi adaptado para o domínio tabular, e os trabalhos correlacionados que motivaram e guiaram a realização deste estudo. Essa revisão teórica fornece o alicerce necessário para a compreensão dos métodos e abordagens utilizados nos capítulos subsequentes.

2.1 Ataque de Negação de Serviço Distribuído

Um DDoS volumétrico é uma tentativa de tornar um serviço indisponível para seus usuários legítimos ao sobrecarregar o alvo com um fluxo massivo tráfego (Mirkovic; Reiher, 2004). Este tráfego excessivo é gerado por múltiplos dispositivos eletrônicos comprometidos, também conhecidos como *bots*, que estão conectados à Internet, com o alvo podendo ser um servidor ou qualquer outro recurso de rede inundado com mensagens, solicitações de conexão ou pacotes malformados (Mirkovic; Reiher, 2004). Esse volume excessivo de dados força o sistema de destino a consumir seus recursos computacionais intensamente, o que pode levar à degradação do desempenho ou até mesmo ao colapso total do sistema (El-Sofany; El-Seoud; Taj-Eddin, 2019).

O ataque DDoS é uma das ameaças mais prejudiciais na Internet atual devido à sua capacidade de interromper sistemas essenciais (El-Sofany; El-Seoud; Taj-Eddin, 2019). Criminosos frequentemente oferecem esse tipo de ataque como um serviço no mercado clandestino, disponível para qualquer indivíduo ou organização que deseje sabotar instituições, sejam elas públicas ou privadas (Hill, 2023). As motivações para esses ataques são diversas, incluindo vingança, prestígio, ganho material e razões políticas, conforme descrito por (Mirkovic; Reiher, 2004). Em muitos casos, o alvo aparente do ataque pode não ser a verdadeira vítima; outras entidades que dependem do funcionamento correto do alvo podem sofrer as consequências indiretas do ataque.

Além dos impactos diretos, os ataques DDoS podem ser usados como distração para encobrir outras atividades maliciosas, como invasões de sistemas ou roubo de dados (Mathews, 2016). Os atacantes aproveitam a confusão e os recursos desviados para a mitigação do problema para realizar ações mais discretas e potencialmente mais danosas.

Esse fator aumenta ainda mais a complexidade e a gravidade da ameaça representada pelos ataques DDoS, exigindo estratégias robustas e multifacetadas para a defesa eficaz contra tais incursões cibernéticas (El-Sofany; El-Seoud; Taj-Eddin, 2019).

2.1.1 Contramedidas aos ataques de DDoS

Assim como outras ameaças cibernéticas, os ataques DDoS volumétricos podem ser mitigados por diversos mecanismos de defesa (Zargar; Joshi; Tipper, 2013). Entre as principais abordagens estão a Prevenção, a Detecção, a Reação e a Predição, cada uma desempenhando um papel específico na mitigação de riscos. Juntas, essas estratégias permitem que sistemas e redes estejam melhor preparados para identificar, mitigar, responder e prever a essas ameaças eficazmente.

2.1.1.1 Prevenção

As medidas preventivas visam minimizar a superfície de ataque, protegendo os alvos potenciais, como sistemas operacionais, protocolos e aplicativos, de serem explorados em ataques de DDoS (Douligeris; Mitrokotsa, 2004). Dentre as várias estratégias, destacam-se a implementação de múltiplas camadas de segurança para evitar que sistemas computacionais sejam comprometidos através da exploração de vulnerabilidades ou acessos não autorizados, e a adoção das melhores práticas de segurança durante o desenvolvimento de software e protocolos de comunicação. Essas medidas incluem a aplicação regular de *patches* de segurança, a configuração correta de *firewalls* e sistemas de detecção de intrusões, e a realização de auditorias de segurança frequentes. Embora essas práticas aumentem significativamente a complexidade para a realização de um ataque, as medidas preventivas por si só não são suficientes para impedir totalmente os ataques de DDoS, pois seria necessário corrigir todas as vulnerabilidades de todos os *hosts* vulneráveis na Internet, o que é uma tarefa praticamente impossível. Por isso, a prevenção deve ser combinada com outras estratégias de defesa para uma proteção mais eficaz.

2.1.1.2 Detecção

A detecção de ataques DDoS é o processo de identificar, em tempo real, padrões de tráfego anômalos que indicam atividades maliciosas destinadas a sobrecarregar sistemas, redes ou serviços. Existem diferentes abordagens para a detecção, que podem ser classificadas em três principais categorias: baseadas em assinaturas, baseadas em anomalias e híbridas (Prakash et al., 2016). Os métodos baseados em assinaturas dependem de padrões previamente conhecidos de ataques registrados em bases de dados, sendo eficazes para ataques já catalogados, mas limitados para identificar novas ameaças. Por outro lado, as abordagens baseadas em anomalias monitoram o tráfego de rede para identificar desvios de comportamento normal, o que permite detectar ataques desconhecidos, mas

pode gerar altos índices de falsos positivos (Dehkordi; Soltanaghaei; Boroujeni, 2021). Abordagens híbridas combinam as vantagens de ambos os métodos, buscando maior precisão e eficiência.

O desafio dos mecanismos de detecção é minimizar a quantidade de falsos positivos, pois modelos de detecção imprecisos podem sobrecarregar o sistema com diagnósticos incorretos, aumentando o consumo de recursos e dificultando a resposta ao incidente (García-Teodoro et al., 2009). Além disso, a detecção eficaz requer a capacidade de distinguir entre tráfego legítimo e tráfego malicioso em tempo real, o que pode ser particularmente desafiador em ambientes de rede dinâmicos e de alto tráfego. Sistemas de detecção que geram muitos falsos positivos não apenas consomem mais recursos, mas também podem levar a uma perda de confiança nos mecanismos de segurança, resultando em respostas inadequadas a ataques reais. Por outro lado, a detecção precisa e eficiente deve equilibrar a sensibilidade e a especificidade para identificar ataques de DDoS com alta precisão, enquanto minimiza o impacto sobre o desempenho geral da rede.

2.1.1.3 Reação

Os mecanismos reativos são projetados para lidar com ataques DDoS em tempo real, buscando mitigar seus efeitos e rastrear os responsáveis pelo ataque. Dentre as estratégias reativas, a reconfiguração da rede é uma abordagem importante, pois ajusta a topologia da rede da vítima para suportar o aumento de requisições durante o ataque. Isso pode incluir a adição ou remoção de recursos, como em escalonamento automático, e o isolamento de máquinas comprometidas (Mirkovic; Prier; Reiher, 2002). Outra técnica crucial é a filtragem de pacotes, que controla o tráfego de entrada e saída para bloquear ou permitir pacotes com base em suas características (Ranjan et al., 2009). A implementação de *rate-limiting* também é utilizada para limitar o volume de tráfego malicioso, garantindo que não exceda a taxa normal gerada por usuários legítimos, e assim evitar a sobrecarga dos sistemas (Yogesh Patil; Ragha, 2011). Além disso, o armazenamento de registros de eventos é essencial para rastrear os endereços IP das máquinas comprometidas, facilitando a identificação das fontes do ataque.

No entanto, a eficácia desses mecanismos pode ser limitada pela rapidez com que um ataque DDoS pode se desenvolver, deixando um curto período para os sistemas implementarem uma resposta eficaz (Singh; Jain, 2024). Ademais, a utilização dessas soluções pode aumentar significativamente o custo da infraestrutura de rede, especialmente no caso de escalonamento automático na nuvem, onde os custos são baseados na quantidade de recursos consumidos (Bremler-Barr; Brosh; Sides, 2017). Portanto, apesar das diversas opções de defesa reativa, muitas vezes elas não são suficientes para neutralizar completamente os danos causados por um ataque DDoS.

2.1.1.4 Predição

A predição de ataques DDoS é uma área focada em técnicas que geram evidências de ataques futuros que podem variar de segundos até dias (Mirkovic; Reiher, 2004); a evidência pode ser uma mensagem de alerta, a probabilidade de ocorrência de um ataque ou qualquer artefato direcionado aos administradores de rede que possa representar um ataque DDoS que o invasor ainda não lançou (Abdlhamed et al., 2017). Modelos de *Machine Learning* são amplamente utilizados para essa finalidade, empregando algoritmos como redes neurais, árvores de decisão e máquinas de vetor de suporte (SVM) (Neira; Kantarci; Nogueira, 2023). Esses modelos são treinados com grandes volumes de dados de tráfego de rede, permitindo a identificação de padrões anômalos que indicam a iminência de um ataque DDoS. Por exemplo, em (H. Salemi H. ROSTAMI, 2022), os autores propõem uma solução para prever ataques DDoS em ambientes de saúde, a solução emite um alerta prevendo um ataque DDoS quando o valor de um determinado expoente é positivo.

Além disso, a pode-se utilizar de modelos estatísticos combinando o *Autoregressive Integrated Moving Average* (ARIMA) e o *Adaptive Conditional Score* (ACS) para prever o *throughput* (taxa de pacotes por minuto da rede) (Adegboyega, 2015). Mesmo que esta solução não seja específica para prever ataques DDoS, os resultados obtidos podem criar evidências próximas ao lançamento de um ataque DDoS.

2.2 Transformers

Transformers são uma arquitetura de modelo de aprendizado de máquina desenvolvida originalmente para processar dados sequenciais, com destaque para tarefas de processamento de linguagem natural (NLP). Introduzida por Vaswani et al. em 2017 no trabalho *Attention Is All You Need* (Vaswani et al., 2017), a arquitetura baseia-se em mecanismos de atenção, que capturam dependências de longo alcance entre os elementos dos dados de entrada, sua arquitetura pode ser vista na Figura 1. Diferentemente das Redes Neurais Recorrentes (RNNs) e Convolucionais (CNNs), os *Transformers* não processam dados de forma sequencial, permitindo maior eficiência em termos de paralelismo durante o treinamento e a inferência. Em geral, esse processamento não sequencial não impactou negativamente os resultados da adoção dos *tranformers* para detecção de ataques DDoS.

A principal inovação dos *Transformers* é o mecanismo de atenção autorregressiva, que avalia a importância de diferentes partes dos dados de entrada para a predição atual, facilitando a captura de relações contextuais complexas em dados textuais e não textuais. Essa capacidade de modelar relações complexas tem levado à sua aplicação em uma ampla gama de tarefas além do NLP, incluindo tradução de idiomas, geração de texto, e mais recentemente, análise de séries temporais e detecção de anomalias em dados tabulares, estes últimos são características que se utiliza neste trabalho. A arquitetura deste modelo

é composta por duas partes principais: o codificador (do inglês, *encoder*) e o decodificador (do inglês, *decoder*). Ambos são empilhamentos de múltiplas camadas idênticas.

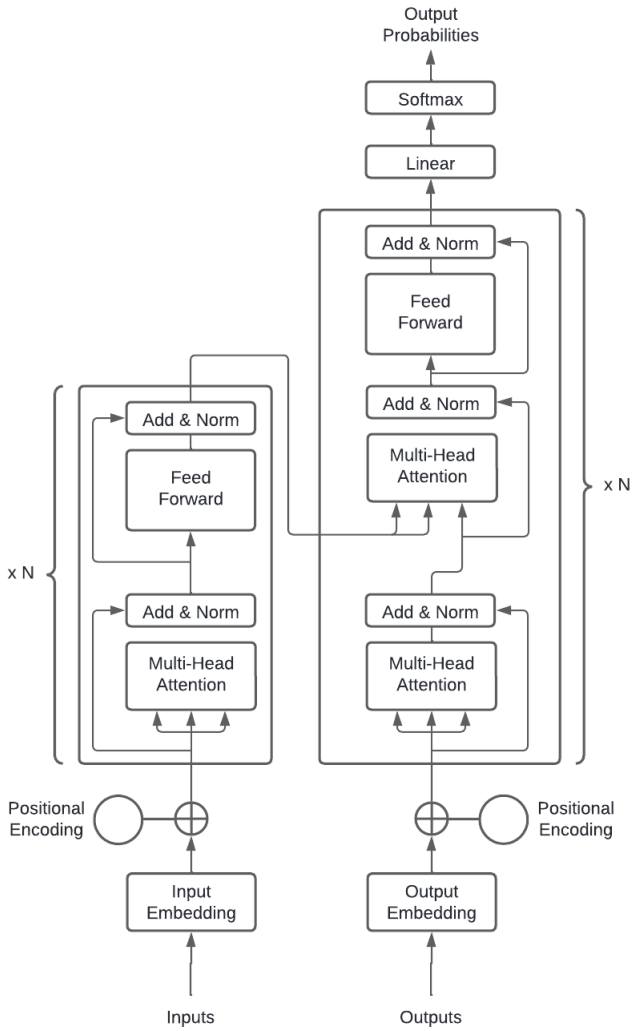


Figura 1 – Ilustra a arquitetura do *Transformer* original, que consiste em um codificador e um decodificador, cada um formado por várias camadas idênticas. O codificador processa a entrada através de camadas de *Multi-Head Attention* e *Feed-Foward*. O decodificador segue um processo similar, mas inclui uma atenção cruzada para integrar as informações do codificador. As conexões residuais e a normalização de camada são aplicadas em cada bloco. Figura baseada em (Vaswani et al., 2017)

2.2.1 Codificador

O codificador é responsável por processar a entrada e transformá-la em uma representação contextual. Ele é formado por uma pilha de N camadas idênticas, cada uma contendo duas subcamadas principais: a camada de *Multi-Head Attention* e a camada de *Feed-Foward*. Essa estrutura permite ao codificador capturar dependências complexas entre

os elementos da entrada, resultando em representações informativas que são fundamentais para o desempenho do modelo.

2.2.1.1 Input Embedding

Para o funcionamento correto do *Transformer*, é essencial realizar o processamento adequado dos dados de entrada. Para alcançar esse objetivo, realiza-se uma representação (do inglês, *embedding*) dos dados de entrada seguida de uma codificação posicional (do inglês, *positional encoding*) de cada informação. Os *Input Embeddings* são representações densas dos *tokens* de entrada (como endereços IP, números de porta, sequências de pacotes, etc.) em um espaço vetorial contínuo. Esse espaço vetorial contínuo indica que, em vez de representar os dados com uma codificação discreta (como números inteiros ou *one-hot encoding*, onde cada *token* ocupa uma posição única), cada *token* é mapeado para um vetor numérico denso, o que permite representar uma gama muito maior de valores possíveis e captura melhor as sutilezas de significado e as relações entre *tokens* de maneira mais precisa. Um exemplo disso pode ser observado na Figura 2.

Cada elemento na sequência de entrada é mapeado para um vetor de dimensão fixa d_{model}. Este mapeamento é aprendido durante o treinamento e os vetores resultantes são chamados de *embeddings*. Dada uma sequência de entrada de *tokens* $[x_1, x_2, \ldots, x_n]$ cada *token* x_i é convertido em um vetor e_i usando uma matriz de representação E:

$$e_i = E(x_i) \tag{2.1}$$

Onde E é uma matriz de *embedding* de tamanho $|V|xd_{model}$, sendo $|V|$ o tamanho do vocabulário e d_{model} a dimensão dos vetores.

2.2.1.2 Positional Encoding

Embora os *embeddings* representem o conteúdo dos *tokens* (unidades discretas de texto, como palavras ou subpalavras), eles não contêm informações sobre a posição dos *tokens* na sequência. Como o *Transformer* não usa arquiteturas recorrentes ou convolucionais, ele precisa de um mecanismo para introduzir informações sobre a ordem dos *tokens*. Esse mecanismo é fornecido pelo codificador posicional (do inglês *Positional Encoding*). *Positional Encodings* são vetores adicionados aos *embeddings* de entrada para incorporar informações de posição. Cada posição na sequência tem um vetor único associado somado ao vetor de *embedding* correspondente. A fórmula utilizada para calcular o *Positional Encoding* é:

$$PE(pos, 2i+1) = cos(\frac{pos}{10000^{2i/dmodel}}) \tag{2.2}$$

$$PE(pos, 2i) = sin(\frac{pos}{10000^{2i/dmodel}}) \tag{2.3}$$

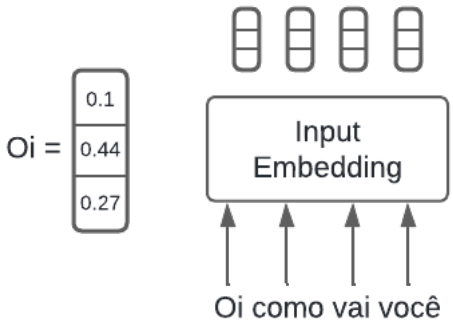


Figura 2 – Exemplo do processo de transformação da frase "Oi como vai você" em *embeddings* de entrada. Cada palavra da frase é dividida em vetores de tamanho 3, com cada posição do vetor contendo um número que representa semanticamente o *token*. No caso da palavra "Oi", por exemplo, ela é convertida em um vetor de tamanho 3, onde cada valor numérico é uma representação densa da informação. Esse processo transforma as palavras em um formato numérico contínuo, permitindo que o modelo capte relações semânticas e sintáticas entre os *tokens*. Figura adaptada de um conjunto de outras imagens da literatura associado ao modelo (Gorishniy et al., 2023).

Onde *pos* é a posição do *token* na sequência, i é a dimensão, d_{model} é a dimensão dos *embeddings*. A formula acima garante que posições diferentes tenham representações diferentes e que posições próximas tenham representações similares (Vaswani et al., 2017). Por fim, os *embeddings* de posição são somados aos *embeddings* de entrada para incorporar a informação posicional:

$$z_i = e_i + P \cdot E_i \tag{2.4}$$

z_i é o vetor de entrada modificado que contém tanto a informação semântica quanto a posicional.

2.2.1.3 Mecanismo de Atenção

O mecanismo de atenção é uma parte fundamental da arquitetura dos *Transformers*, permitindo aos modelos focar em diferentes partes dos dados de entrada de maneira dinâmica e contextual (Vaswani et al., 2017). Esse mecanismo opera mediante um processo de várias etapas, que começa com a transformação dos dados de entrada em três vetores distintos: consultas (*queries*), chaves (*keys*) e valores (*values*). Esses vetores são gerados pela multiplicação dos dados de entrada com matrizes de parâmetros treináveis.

As consultas representam o elemento de entrada para o qual a atenção está sendo calculada, enquanto as chaves representam todos os elementos de entrada que estão sendo considerados para a relevância em relação à consulta. Os valores são as informações

associadas a cada chave, que serão combinadas para formar a saída final. Por exemplo, quando você digita uma consulta para pesquisar algum vídeo no Youtube, o mecanismo de pesquisa mapeia sua **query** em relação a um conjunto de **keys** (título do vídeo, descrição etc.) associadas aos vídeos candidatos no banco de dados e, em seguida, apresenta os melhores vídeos correspondidos **values**.

O próximo passo no mecanismo de atenção envolve o cálculo das pontuações de atenção, feito como o produto escalar entre a consulta e todas as chaves, seguido de uma normalização. Isso é expresso pela fórmula

$$score(Q, K) = \frac{Q \cdot K^T}{\sqrt{d_k}} \tag{2.5}$$

sendo d_k a dimensão dos vetores de chave. A normalização pelo fator $\sqrt{d_k}$ ajuda a estabilizar os gradientes durante o treinamento.

Após calcular as pontuações, estas são passadas por uma função *softmax* para converter as pontuações em probabilidades, determinando o peso relativo de cada valor correspondente às chaves na construção da saída final. A fórmula para a aplicação da *softmax* é:

$$Attention\ weights = softmax(\frac{Q \cdot K^T}{\sqrt{d_k}}) \tag{2.6}$$

A saída do mecanismo de atenção é então uma média ponderada dos valores, onde os pesos são as probabilidades calculadas pela função *softmax*, expressa como

$$Attention\ weights(Q, K, V) = softmax(\frac{Q \cdot K^T}{\sqrt{d_k}})V \tag{2.7}$$

Para capturar os diferentes tipos de dependências nos dados, os Transformers utilizam a *Multi-Head Attention*. Isso significa que o processo de atenção é realizado várias vezes em paralelo com diferentes subconjuntos dos vetores de peso, conhecidos como *attention heads*. As saídas de todas as *heads* são então concatenadas e passadas por uma camada linear para combinar a informação de diferentes cabeças, resultando na fórmula.

$$MultiHead\ Attention(Q, K, V) = Concat(head_1, \ldots, head_h)W^O \tag{2.8}$$

Cada $head_i$ é o resultado da atenção individual, e W^O é uma matriz de parâmetros treináveis. Por exemplo, na frase “Oi, como vai você?”, o mecanismo de atenção calcula a similaridade entre a primeira palavra, “Oi”, e todas as outras palavras da sentença. Esse processo é repetido para todas as outras palavras, ou seja, no final tem-se uma matriz de valores representando esse mecanismo de atenção. Um exemplo com valores figurativos é demonstrado na Figura 3.

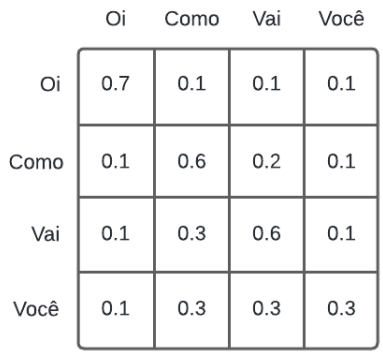


Figura 3 – Mostra uma matriz 4x4 que ilustra como o mecanismo de atenção calcula a similaridade entre as palavras da frase "Oi como vai você". Cada célula da matriz contém um valor que representa o grau de atenção entre os tokens (palavras no caso). Por exemplo, o valor 0.7 na posição (1,1) indica que a palavra "Oi"tem forte atenção sobre si mesma, enquanto valores menores, como 0.1, indicam menor atenção entre palavras diferentes. Figura adaptada de um conjunto de outras imagens da literatura associado ao modelo (Gorishniy et al., 2023).

2.2.1.4 *Feed-Foward Neural Network* (FFNN)

A *Feed-Foward Neural Network* (FFNN) é responsável por introduzir não linearidade e complexidade nos dados processados (Bebis; Georgiopoulos, 1994). A FFNN no *Transformer* é composta por duas camadas lineares (ou densas) separadas por uma função de ativação não linear. O objetivo dessas camadas é transformar a representação dos dados de forma não linear, permitindo que o modelo capture padrões complexos.

1. **Primeira Camada Linear**
   A primeira camada linear transforma a entrada em uma representação de maior dimensão. Se a dimensão da entrada é d_{model} a dimensão intermediária é d_{ff} esta camada aplica uma transformação linear que mapeia um vetor de dimensão d_{model} para um vetor de dimensão d_{ff}:

$$FFNN_out1 = ReLU(X \cdot W_1 + b_1) \tag{2.9}$$

   onde X é a entrada, W_1 são os pesos da camada linear, b_1 é o vetor de bias, e ReLU é a função de ativação.

2. **Função de Ativação**
   A função de ativação usada é tipicamente a ReLU (*Rectified Linear Unit*), definida como:

$$ReLU(x) = max(0, x) \tag{2.10}$$

A ReLU introduz não linearidade na rede, permitindo que ela aprenda representações mais complexas.

3. **Segunda Camada Linear**
   A segunda camada linear transforma a saída da primeira camada de volta para a dimensão original:

$$FFNN_out2 = FFNN_out2 \cdot W_2 + b_2 \tag{2.11}$$

   onde W_2 são os pesos da segunda camada linear e b_2 é o vetor de bias correspondente.

Assim como outras partes do *Transformer*, a FFNN é seguida por uma operação de normalização de camada (do inglês, *layer normalization*) e uma conexão residual (do inglês, *residual connection*). A normalização de camada é aplicada após a FFNN para estabilizar o treinamento e melhorar a convergência do modelo. Ela normaliza as ativações da camada para ter média zero e variância unitária, ajudando a manter as distribuições dos dados consistentes através das camadas. Já a conexão residual adiciona a entrada original da FFNN à sua saída. Isso ajuda a mitigar problemas de *vanishing gradients* (ocorre durante o processo de treinamento da rede, mais especificamente durante a retro-propagação do erro, usada para ajustar os pesos da rede) permitindo que o gradiente flua diretamente pela rede durante o treinamento (Vaswani et al., 2017).

2.2.2 Decodificador

A estrutura do decodificador é composta por várias camadas idênticas (geralmente seis), cada uma contendo três subcomponentes principais: o mecanismo de atenção mascarada multi-cabeça (do inglês, *Masked Multi-Head Self-Attention*), o mecanismo de atenção multi-cabeça (do inglês, *Multi-Head Attention*) e a rede neural *Feed-Foward* (FFNN). O primeiro subcomponente, o *Masked Multi-Head Self-Attention* é responsável por processar as sequências de saída de maneira autoregressiva. Esse mecanismo é similar ao *Self-Attention* do codificador, mas possui uma diferença fundamental: a utilização de uma máscara. Essa máscara é aplicada para garantir que, durante o treinamento, a predição do próximo item em uma sequência dependa exclusivamente dos *tokens* anteriores, impedindo o modelo de acessar informações futuras (Vaswani et al., 2017). Essa característica é essencial para tarefas de geração de sequência, como tradução automática, onde a coerência na construção progressiva da saída é imprescindível (Pelofske; Urias; Liebrock, 2024).

A máscara é implementada atribuindo pesos de atenção iguais a zero para todos os *tokens* futuros em relação à posição atual na sequência. Por exemplo, ao prever o *token* na posição t, o modelo só pode considerar informações até a posição $t-1$, criando uma dependência causal. Essa restrição garante que a saída seja gerada de maneira incremental,

conforme o comportamento esperado para a maioria das tarefas sequenciais (Vaswani et al., 2017).

A Figura 4 ilustra como a máscara é aplicada durante o cálculo das pontuações de atenção. O exemplo demonstra a soma dos escores escalados com o *look-ahead mask*, resultando nos escores mascarados. Esse processo assegura que o treinamento respeite a ordem temporal dos dados e que previsões só dependam de informações passadas.

Portanto, o *Masked Multi-Head Self-Attention* não apenas protege a integridade do processo de geração de sequência, mas também assegura que o treinamento do modelo respeite a ordem temporal dos dados (Soni, 2023). Isso reforça a capacidade do decodificador de realizar previsões coerentes e alinhadas com o contexto dos *tokens* previamente processados, destacando sua importância na arquitetura geral do Transformer.

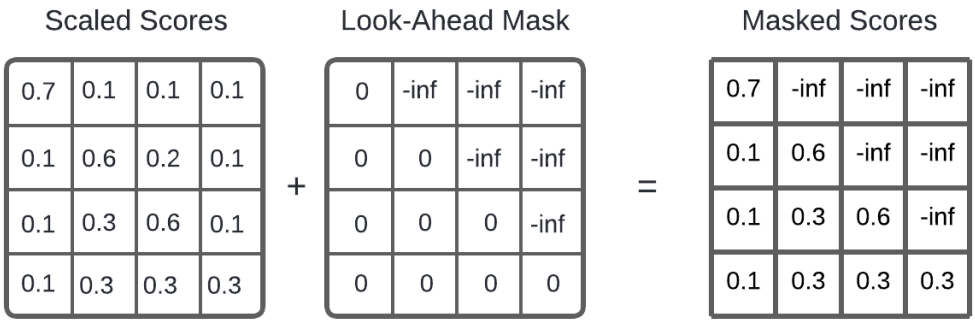


Figura 4 – Exemplo do mecanismo *look-ahead mask*, onde os valores de atenção para posições futuras são mascarados, assegurando que as previsões dependam apenas dos *tokens* já processados. Figura adaptada de um conjunto de outras imagens da literatura associado ao modelo (Gorishniy et al., 2023).

O segundo subcomponente é o *Multi-Head Attention*, que realiza a atenção sobre a saída do codificador. Este mecanismo permite que o decodificador se concentre em partes relevantes da sequência de entrada enquanto gera a sequência de saída. Esta etapa cruza informações da entrada com a sequência de saída gerada até o momento, permitindo que o decodificador considere o contexto completo da entrada enquanto constrói a saída (Vaswani et al., 2017).

O terceiro subcomponente é o *Feed-Foward Neural Network* (FFNN), similar ao codificador, cada camada do decodificador tem uma rede neural *Feed-Foward* aplicada de forma independente a cada posição. Esta rede consiste em duas camadas totalmente conectadas com uma função de ativação não-linear (geralmente ReLU) no meio. A saída do FFNN é então passada por uma camada de normalização antes de ser passada para a próxima camada do decodificador. Essa subcamada aumenta a capacidade de modelagem

do decodificador, permitindo a captura de relações complexas entre os *tokens* (Vaswani et al., 2017).

O fluxo de dados através do decodificador começa com a entrada de vetores de representação e codificações posicionais. A sequência de entrada do decodificador é convertida em representações vetoriais, somadas com as codificações posicionais para incorporar a informação da posição dos elementos na sequência de saída. Em seguida, o *Masked Multi-Head Self-Attention* é aplicado nas codificações posicionais, garantindo que apenas os elementos anteriores influenciem a predição do próximo item (Soni, 2023).

Após o *Masked Multi-Head Self-Attention*, o decodificador aplica o *Multi-Head Attention*, que considera a saída do codificador. Isso permite que o decodificador alinhe a sequência de entrada com a sequência de saída parcial gerada até aquele ponto (Vaswani et al., 2017). Finalmente, a rede neural *Feed-Foward* é aplicada a cada posição individualmente, seguida por uma camada de normalização e uma conexão residual, que auxiliam na estabilização do treinamento e na retenção de gradientes através das camadas profundas do modelo.

2.2.3 Saída

Após o processamento através das várias camadas do codificador e do decodificador do *Transformer*, a saída gerada precisa ser transformada em uma forma que permita a previsão dos próximos *tokens* ou palavras. Para isso, o *Transformer* utiliza um classificador linear seguido por uma função de ativação *softmax*. Primeiro, a saída do decodificador, como visto na subseção anterior, é uma matriz que passa por uma camada linear. Essa camada linear, ou camada densa, transforma cada vetor de alta dimensionalidade em um vetor de dimensionalidade igual ao tamanho do vocabulário do modelo. O tamanho do vocabulário é o número total de *tokens* que o modelo pode prever. Essa transformação é realizada por uma operação de multiplicação de matriz e soma de bias, onde cada vetor de saída x_i é mapeado para um novo vetor $W \cdot x_i + b$, sendo W a matriz de pesos da camada linear e b o vetor de bias.

Após a camada linear, a saída passa por uma função de ativação *softmax*. A função *softmax* é usada para converter os valores do vetor resultante (que podem ser arbitrários) em probabilidades. Essa função calcula a exponencial de cada valor no vetor e, em seguida, normaliza esses valores dividindo pela soma de todas as exponenciais. Esse processo permite que o modelo atribua uma probabilidade a cada *token* no vocabulário, facilitando a escolha do próximo *token* na sequência. A Figura 5 ilustra o fluxo do classificador linear, desde os vetores de saída até as probabilidades geradas pela função *softmax*.

2.3 FT-Transformer

No artigo *Revisiting Deep Learning Models for Tabular Data* (Gorishniy et al., 2023), os autores apresentam uma adaptação simples da arquitetura *Transformer* (Vaswani

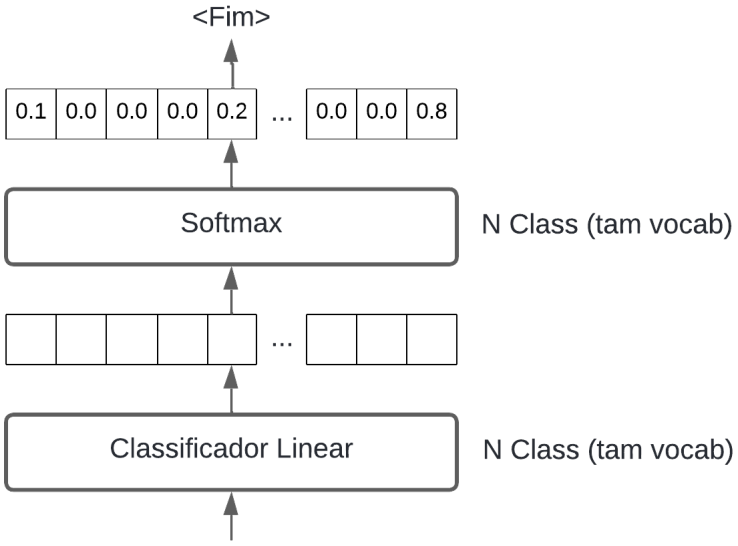


Figura 5 – Fluxo da saída no Transformer: a transformação dos vetores gerados pelo decodificador em probabilidades, utilizando o classificador linear seguido pela função *softmax*. Figura adaptada de um conjunto de outras imagens da literatura associado ao modelo (Gorishniy et al., 2023).

et al., 2017) para o domínio tabular. A Figura 6 demonstra as principais partes do *FT-Transformer*. Resumindo, através do uso do *feature tokenizer* este modelo transforma todos os dados (categóricos e numéricos) em *embeddings* e aplica uma pilha de camadas *Transformer* a esses novos dados. Assim, cada camada do *Transformer* opera no nível de recurso de um objeto.

Este modelo foi a escolha final para ser utilizado neste TCC, pois além de desenvolvido especificamente para trabalhar com dados tabulares, comuns em muitos problemas de aprendizado de máquina, ele também aplica a normalização de *features* diretamente nos *embeddings*, importante para os dados tabulares que possuem escalas diferentes. Isso ajuda a estabilizar o treinamento e a melhorar o desempenho geral do modelo (Gorishniy et al., 2023). Assim, ele consegue capturar interações complexas entre diferentes informações de uma maneira que os modelos tradicionais de aprendizado profundo (como MLPs ou mesmo *Transformers* originais) podem não capturar tão bem.

Outro motivo específico para essa escolha é devido aos dados utilizados para os testes, o *dataset* disponibilizado por (Stratosphere, 2015) contem dados apenas numéricos, que são difíceis de serem processados corretamente de um modo satisfatório pelos outros modelos de *Transformer*. Já o *FT-Transformer* possui a capacidade de converter os dados em categóricas, podendo assim serem passados pela estrutura do *Transformer*.

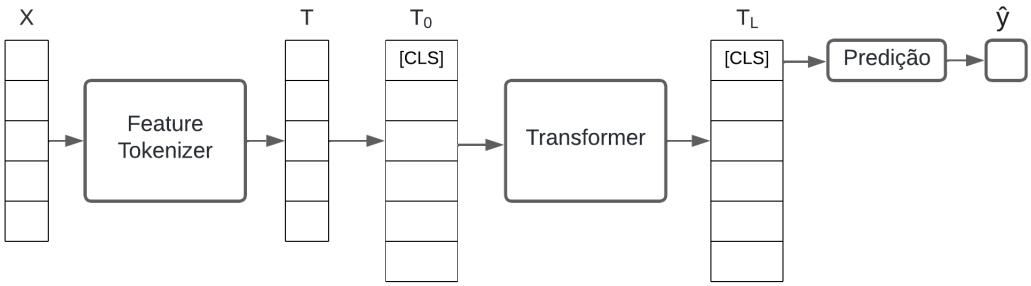


Figura 6 – A arquitetura do *FT-Transformer*. Em primeiro lugar, o *Feature Tokenizer* transforma dados em *emdeddings*. Estes são então processados pelo módulo *Transformer* e a representação final do *token* [CLS] é usada para previsão. Figura baseada em (Gorishniy et al., 2023)

2.3.1 Feature Tokenizer

O *Feature Tokenizer* começa mapeando cada característica tabular para um vetor de dimensão fixa. Para características categóricas, utiliza-se uma matriz de *embeddings* onde cada categoria é mapeada para um vetor denso. Para características numéricas, são aplicadas técnicas de discretização ou *embeddings* contínuos para transformá-las em vetores. Este processo cria representações vetoriais uniformes que o modelo pode manipular, independentemente do tipo de dado original. A Figura 7 ilustra esse processo, mostrando como os diferentes tipos de características são processados e combinados.

Essas representações são então combinadas e somadas com as codificações posicionais (do inglês *positional encodings*), essenciais para manter a ordem das características. Esta etapa assegura que o modelo entenda não apenas os valores das características, mas também suas posições relativas, uma informação crucial para capturar dependências nos dados tabulares. Uma vez que as características são "tokenizadas" e posicionadas, elas passam pelos componentes do *Transformer* que foram mencionados na seção anterior. As fórmulas para o cálculo do *Tokenizer* são:

$$T_j{}^{(num)} = b_j{}^{(num)} + x_j{}^{(num)} \cdot W_j^{(num)} \tag{2.12}$$

$$T_j{}^{(cat)} = b_j{}^{(cat)} + e_j{}^{T} \cdot W_j{}^{(cat)} \tag{2.13}$$

$$T = stack[T_1{}^{(num)}, \ldots, T_{k^{(num)}}{}^{(num)}, T_1{}^{(cat)}, \ldots, T_{k^{(cat)}}{}^{(cat)} \tag{2.14}$$

onde $e_j{}^T$ é um vetor one-hot para o recurso categórico correspondente, b_j é a j-th *feature bias*, $f_j{}^{num}$ é implementado como a multiplicação elemento a elemento com o vetor $W_j{}^{num} \in \mathbb{R}^d$ e $f_j{}^{(cat)}$ é implementado como a tabela de pesquisa $W_j{}^{(cat)} \in \mathbb{R}^{S_j \times d}$ para entradas categóricas.

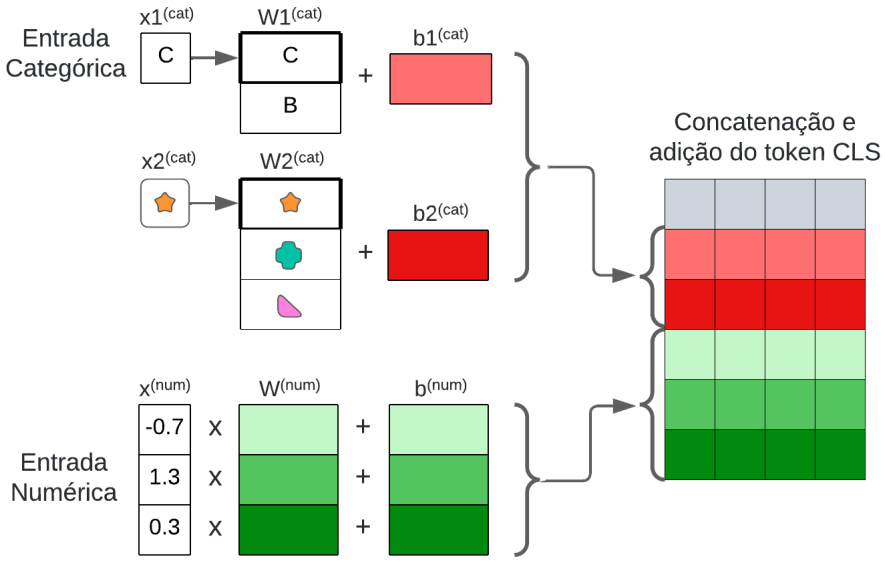


Figura 7 – Processo do *Feature Tokenizer*, ilustrando a transformação de duas características categóricas e três características numéricas em representações vetoriais uniformes. Essas representações são concatenadas e recebem a adição do *token* [CLS] para serem processadas pelo modelo. Baseado em (Gorishniy et al., 2023).

2.3.1.1 Token CLS

O [CLS] é um *token* especial inserido no início de cada sequência de entrada. Sua inclusão serve como ponto de referência para a agregação de informações da sequência inteira (Gorishniy et al., 2023). Durante o processo de atenção dentro do *Transformer*, o *token* [CLS] coleta informações contextuais de todos os outros itens da sequência. Isso é feito através dos mecanismos de *self-atention* que permitem que o [CLS] incorpore representações ricas e contextualmente informadas da sequência completa. O vetor de saída associado ao [CLS] é utilizado como uma representação agregada da sequência inteira. Esse vetor é então alimentado em uma camada de classificação linear seguida por uma função de ativação *softmax* para realizar a tarefa de classificação.

2.4 Trabalhos correlacionados

No TCC1, foi realizada uma busca pelos modelos de *Transformer* mais adequados para a predição de ataques DDoS utilizando dados tabulares. Durante esse processo, destacou-se o *TabTransformer*, proposto por (Huang et al., 2020), que aplica mecanismos de atenção para capturar interações entre características tabulares de maneira eficiente. Apesar de seu desempenho superior em tarefas de classificação e regressão, o modelo não utiliza o *Transformer* diretamente para dados numéricos, processando-os fora do mecanismo de atenção, como se pode ver na Figura 8. Essa limitação tornou-o inadequado para os

objetivos do estudo, que requeriam um modelo capaz de integrar atributos numéricos e categóricos de forma nativa.

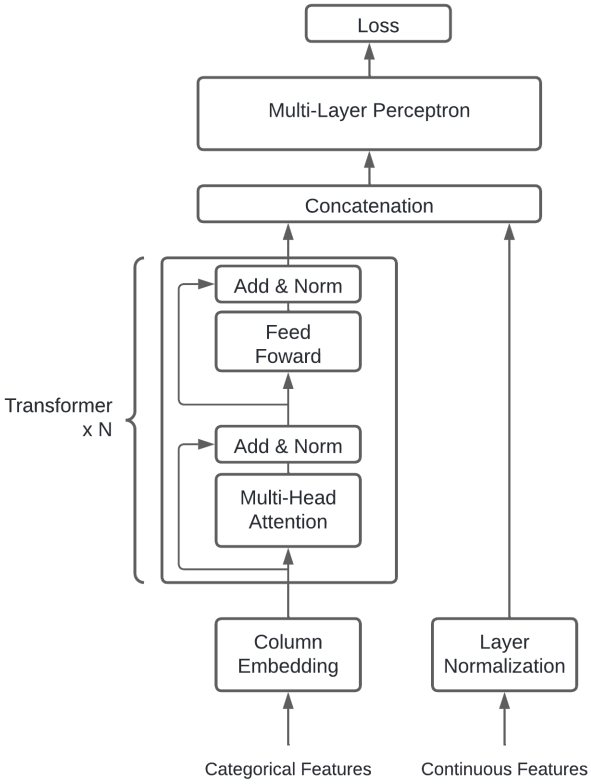


Figura 8 – Arquitetura do *TabTransformer*, que utiliza mecanismos de atenção para processar interações entre características categóricas, enquanto os atributos numéricos são processados separadamente fora do mecanismo de atenção. Baseado em (Huang et al., 2020).

Outro modelo analisado foi o *SparseTransformer*, introduzido por (Child et al., 2019). Esse modelo foca na eficiência computacional, restringindo a atenção a uma porção reduzida de tokens. Embora seja útil para sequências longas, como textos ou séries temporais, ele não ofereceu vantagens significativas ao lidar com dados tabulares, que carecem da estrutura sequencial clara presente nesses domínios.

Por essas razões, optou-se pelo *FT-Transformer* como modelo central para o trabalho, já que ele se destaca por integrar atributos numéricos e categóricos em dados tabulares por meio de tokenização. (Gorishniy et al., 2023) demonstraram que o modelo supera métodos tradicionais, como árvores de decisão e redes neurais profundas, em termos de precisão e robustez, consolidando-se como uma abordagem eficaz para dados tabulares.

Estudos recentes destacam a aplicabilidade do *FT-Transformer* em diferentes domínios. (Praveen et al., 2024) empregaram essa arquitetura para detecção de intrusões em redes (*Network Intrusion Detection Systems - NIDS*), demonstrando desempenho superior ao de abordagens tradicionais em tarefas com dados tabulares de tráfego de rede. A pesquisa

evidenciou sua capacidade de capturar relações complexas e interdependências em cenários de alta variabilidade. De maneira semelhante, (Zhang; Zhang, 2024) integraram o *FT-Transformer* a um mecanismo de detecção de mudanças conceituais (em inglês, *concept drift*) baseado em KSWIN, resultando em uma abordagem de aprendizado online adaptada a variações contínuas nos dados. Essa solução alcançou precisão média de 99,46% no *dataset* CIC-IDS e 96,79% no NetFlow V2.

Em outro domínio, (Kim et al., 2023) utilizaram o *FT-Transformer* para prever a readmissão de pacientes com insuficiência cardíaca. Nesse contexto, o modelo processou eficientemente atributos numéricos e categóricos, superando métodos tradicionais, como regressão logística e *random forest*. O estudo demonstrou que o modelo consegue identificar padrões sutis em dados médicos, oferecendo um caminho promissor e valioso para a tomada de decisões clínicas.

Apesar dessas contribuições, devido à sua recente introdução (2023), ainda existem poucos trabalhos acadêmicos que utilizam o *FT-Transformer* de maneira robusta e sistemática, principalmente na área de DDoS. Essa limitação ressalta a importância de explorar sua aplicação em diferentes contextos, como a predição de ataques DDoS. O modelo mostrou-se como o sistema mais alinhado aos objetivos deste trabalho, combinando capacidade preditiva com uma arquitetura otimizada para dados tabulares, permitindo avanços significativos na construção de sistemas de segurança mais eficazes.

3 Metodologia

Como mencionado no primeiro capítulo, o objetivo deste trabalho é estudar os ataques DDoS e a estrutura do *FT-Transformer*, visando implementá-lo para a predição de ataques de negação de serviço distribuído, utilizando dados tabulares como entrada. A intenção era verificar a eficiência desses modelos na detecção precoce desses ataques utilizando-se de um *dataset* mais robusto. Através dessa abordagem, espera-se contribuir para o desenvolvimento de sistemas de segurança mais robustos e eficazes. A revisão conduzida no artigo (Neira; Kantarci; Nogueira, 2023) não identificou, na literatura existente, trabalhos que aplicassem modelos de *Transformers* à predição de ataques DDoS. Em vista dessa lacuna, decidiu-se elaborar um estudo a partir de uma abordagem inicial, utilizando como referência exclusivamente os dados disponibilizados por (Neira; Borges et al., 2023) e o modelo *FT-Transformer*, proposto por (Gorishniy et al., 2023).

Ao longo do desenvolvimento deste trabalho, foram realizadas diversas etapas para garantir a robustez e a precisão do modelo implementado. Inicialmente, foi feita uma seleção criteriosa dos dados, focando em capturas de tráfego de rede que contivessem características típicas de ataques DDoS volumétricos. Em seguida, o modelo *FT-Transformer* foi treinado e ajustado com diferentes parâmetros para otimizar seu desempenho. As métricas de avaliação, como precisão, *recall*, F1-score e matriz de confusão foram utilizadas para medir a eficácia do modelo na predição de ataques. As seções a seguir detalham esse processo.

3.1 Conjunto de Dados

Este trabalho realiza treze experimentos para avaliar o *FT-Transformer*. Os testes consideram três conjuntos de dados com características diferentes. Estes conjuntos de dados incluem ataques DDoS em redes internas e redes onde os *bots* se ligam à vítima através da Internet. Além disso, possuem diferentes números de *bots*, diferentes tamanhos de tráfego de rede e vários dispositivos normais ativos.

Utilizando os três *datasets*, foram conduzidos um total de 13 experimentos. A configuração para a execução dos experimentos utilizou a máquina T4 disponibilizada pelo Google Colab. Essa máquina foi escolhida devido à sua configuração, que oferece um equilíbrio entre desempenho e custo, sendo adequada para os requisitos do modelo. A T4 conta com um processador Intel Xeon de 2,20 GHz, 16 GB de RAM e uma GPU NVIDIA Tesla T4, que possui 16 GB de memória de vídeo GDDR6. Essa configuração forneceu os recursos computacionais necessários para treinar e ajustar o modelo, garantindo eficiência no processamento e no manuseio dos dados.

O objetivo de todos os experimentos foi prever se o fluxo de dados era gerado por um *bot* (1) ou não (0). O *FT-Transformer* poderia ser treinado até 100 épocas, porém,

diferentes critérios de parada foram utilizados, interrompendo o treinamento antes disso, dentre eles: a ausência de melhoria na métrica de validação por um número pré-definido de épocas consecutivas, conhecido como *early stopping*, e o limite de iterações estipulado para evitar sobreajuste. Esses critérios garantiram que o modelo não continuasse treinando desnecessariamente em condições de estagnação, otimizando o uso de recursos computacionais e melhorando a eficiência do processo de treinamento. Além disso, foi utilizado 15 sementes de números aleatórios conforme sugerido por (Gorishniy et al., 2023) e o restante das métricas se mantiveram as mesmas do paper do modelo original.

A Tabela 1 apresenta os 51 atributos utilizados nos experimentos. Estes atributos são baseados nos campos de protocolo do enlace, rede e camadas de transporte do modelo TCP/IP. O *FT-Transformer* não utiliza dados de carga útil de pacote. Assim, apenas os dados do cabeçalho foram considerados nestes experimentos, pois o cabeçalho dos pacotes contém informações estruturais fundamentais, como endereços de origem e destino, portas utilizadas, tipos de protocolo, e indicadores de fluxo que são cruciais para identificar padrões de tráfego anômalo associados a ataques DDoS. Os 51 atributos são extraídos e classificados por intervalos de tempo de um segundo para todas as experiências. Estes valores foram definidos para se adaptarem às características dos conjuntos de dados e para fornecer uma avaliação de diferentes valores de intervalo de tempo.

Atributos	**Descrição**
1–2	Total de endereços MAC de origem/destino únicos
3	Versão mais frequente do protocolo que estará no payload
4–5	Menor/maior tamanho de pacote
6	Soma dos tamanhos dos pacotes
7–9	Total de pacotes dos tipos ICMP/UDP/TCP
10–11	Total de endereços IP de origem/destino únicos
12–16	Maior/menor/soma/STD/média do tempo de vida dos pacotes
17	Total de pacotes transmitidos na rede
18–19	Porta de origem/destino mais frequente nos pacotes
20	Total de flags TCP unicas
21–26	Total de flags TCP FIN/SYN/RST/PUSH/ACK/URG
27–31	Maior/menor/soma/STD/média do campo de tamanho da janela TCP
32–36	Maior/menor/soma/STD/média do campo Número de sequência TCP
37–46	Maior/Menor/Soma/STD/Média do tempo de chegada entre quaisquer pacotes e pacotes TCP
47–51	Maior/Menor/Soma/STD/Média do tempo de chegada do primeiro pacote TCP

Tabela 1 – Atributos iniciais dos conjuntos de dados. Tabela baseada em (Neira; Borges et al., 2023).

O trabalho de (Neira; Borges et al., 2023) marca os intervalos de tempo como normais e maliciosos para quantificar os resultados. Os intervalos maliciosos compreendem intervalos de tempo em que os *bots* enviam pelo menos um pacote. Assim, estes intervalos representam a preparação para o ataque, uma vez que os *bots* impactaram o tráfego da rede antes do lançamento do ataque. Os intervalos normais são intervalos de tempo em que os *bots* não enviam pacotes.

Por fim, então (Neira; Borges et al., 2023) realiza a engenharia de sinal após recolher os 51 atributos apresentados na Tabela 1. A engenharia de sinal aplica AC-1, AC-2, AC-3, Skewness, Kurtosis e CV a todos os dados recolhidos para criar 306 novas features a cada intervalo de um segundo, num total de 664 features. Essas novas características são a base para todos os passos seguintes realizados pelo *FT-Trasformer*.

O primeiro indicador, o Skewness, mede a assimetria da distribuição dos dados em relação à média. Na fórmula do Skewness Eq. 3.1, x_t representa cada elemento da série temporal, N é a quantidade total de valores coletados, $\overline{x}$ é a média aritmética e σ é o desvio padrão Eq. 3.2. A variação na assimetria dos dados pode servir como um alerta precoce confiável.

$$Skewness = \frac{N}{(N-1)(N-2)} \sum_{t=1}^{N} (\frac{x_t - \overline{x}}{\sigma})^3 \tag{3.1}$$

$$\sigma = \sqrt{\frac{1}{N} \sum_{t=1}^{N} (x_t - \overline{x})^2} \tag{3.2}$$

O segundo indicador, o Kurtosis, mede o grau de achatamento da distribuição dos dados. Na fórmula do Kurtosis Eq. 3.3, $N, x_t, \overline{x}$ representam, respectivamente, a quantidade total de dados, cada valor coletado e a média dos dados. O $\hat{a}$ é necessário para o cálculo. Valores positivos indicam uma distribuição com pico, negativos indicam uma distribuição plana e valores próximos a zero indicam uma distribuição normal. Variações no Kurtosis podem ser sinais precoces de ataques DDoS.

$$Kurtosis = \frac{N(N+1}{(N-1)(N-2)(N-3)} \sum_{t=1}^{N} (\frac{x_t - \overline{x}}{\sigma})^4 - \frac{3(N-1)^2}{(N-2)(N-3)} \tag{3.3}$$

$$\frac{1}{N} \sum_{t=1}^{N} (x_t - \overline{x})^2 \tag{3.4}$$

O terceiro indicador, o coeficiente de variação (CV) Eq. 3.5, é obtido dividindo-se o desvio padrão (σ) pela média ($\overline{x}$). Este indicador mostra a variação dos dados em relação à média e pode indicar transições críticas futuras.

$$Coeficiente\ de\ Variação = \frac{\sigma}{\overline{x}} \tag{3.5}$$

Os últimos três indicadores, AC-1, AC-2 e AC-3, também chamados de Autocorrelação (AC) Eq. 3.6, medem a correlação de uma série temporal consigo mesma em diferentes

atrasos, elas referem-se à autocorrelação nos atrasos 1, 2 e 3, respectivamente. Estes indicadores ajudam a identificar padrões cíclicos e a força da relação entre os valores atuais e passados na série temporal. Nele, x_t é valor temporal da série no tempo t, x é a média da série temporal, N é o número total de observações, K é o lag (atraso) para o qual se está calculando a autocorrelação.

$$\mathrm{AC}_k = \frac{\sum_{t=1}^{N-k}(x_t - \bar{x})(x_{t+k} - \bar{x})}{\sum_{t=1}^{N}(x_t - \bar{x})^2} \tag{3.6}$$

3.2 Métricas para avaliação

Para avaliar a eficácia do sistema, este trabalho utiliza as métricas de acurácia, precisão, *recall* e F1-score. Todas elas são calculadas tanto para intervalos normais quanto para intervalos maliciosos. A precisão mede a proporção de predições corretas de ataques maliciosos em relação a todas as predições feitas para essa classe, enquanto o *recall* avalia a proporção de ataques maliciosos corretamente identificados em relação ao total real de ataques maliciosos. O F1-score, por sua vez, é a média harmônica entre a precisão e o *recall*, proporcionando um equilíbrio entre essas métricas, especialmente útil em cenários com classes desbalanceadas. A detecção de ataques DDoS lida com o desafio do grande desbalanceamento das classes, com uma predominância de intervalos normais sobre os maliciosos. Para simplificar a apresentação dos resultados e considerar esse desbalanceamento, são apresentadas as médias ponderadas de precisão e *recall*, considerando o total de intervalos maliciosos e normais. Nas equações abaixo, FP representa o total de falsos positivos, FN é o total de falsos negativos, VP indica o total de verdadeiros positivos, e VN indica o total de verdadeiros negativos.

$$Acuracia = \frac{VP + VN}{VP + FN + VN + FP} \tag{3.7}$$

$$Precisão = \frac{TP}{TP + FP} \tag{3.8}$$

$$Recall = \frac{TP}{TP + FN} \tag{3.9}$$

$$F1 = 2 \times \frac{Precison \times Recall}{Precision + Recall} \tag{3.10}$$

3.3 Transformação e preparação dos dados

Para conduzir este experimento foi necessário aplicar uma transformação no formato dos dados originais. Os dados disponibilizados por (Neira; Borges et al., 2023) estão contidos em um .CSV, o que é incompatível com o programa em Python do autor (Gorishniy et al., 2023), por isso foi necessário convertê-los no formato .numpy. Além disso, diferente

de (Neira; Borges et al., 2023) que utilizaram um terço dos dados para treinamento, foi removida primeiras entradas que não possuíam dados e o restante foi divido para teste e validação dependendo do experimento e cenário.

Os experimentos realizados por (Neira; Borges et al., 2023) utilizaram como atributos o total de endereços IP de origem e destino, o total de pacotes trafegados em cada intervalo de tempo, dentre outros. Tais atributos foram escolhidos, pois segundo (Feng et al., 2018) o total de pacotes é um atributo crucial na identificação do tráfego de comando e controle, que geralmente ocorre antes do início de um ataque DDoS e de acordo com (Jyoti; Behal, 2021) a falsificação de IPs é uma ação comum em ataques DDoS. Além disso, esses atributos podem ser extraídos apenas utilizando os cabeçalhos dos pacotes, facilitando a captura e o armazenamento dos dados, pois não há necessidade de coletar a carga útil dos pacotes.

4 Resultados

Nas próximas subseções são apresentados os resultados para os 13 experimentos. O melhor desempenho foi observado na primeira época com a semente de número um. Assim, todos os gráficos e dados apresentados a seguir referem-se a essa configuração específica. São apresentados gráficos que comparam os *logits* gerados pelo *FT-Transformer* com os valores de saída dos *datasets* originais. *Logits*, nesse contexto, são os valores brutos de saída do modelo antes de serem transformados por uma função de ativação, como a *Sigmoid* ou *Softmax*. Para este caso utilizou-se o *softmax* como limiar de 0.5, ou seja, valores acima foram contados como 1 e abaixo como 0. Esses valores representam as evidências não normalizadas para cada classe, ou seja, quão fortemente o modelo associa uma amostra a uma determinada classe. Por exemplo, para uma classificação binária, o *logit* é um único valor que indica a "força" da evidência para a classe 1. A análise desses valores permite compreender melhor como o modelo chega às suas decisões e identificar possíveis padrões ou discrepâncias em relação aos resultados esperados.

4.1 Experimentos com conjunto de dados 1

O conjunto de dados 1 refere-se à captura 51, desenvolvida pela Universidade Técnica Checa (CTU) e denominada CTU-13 (García et al., 2014). Esta captura conta com 10 *bots* ativos na rede, 41 GB de dados, 46.997.342 pacotes, 8.803 segundos e ataques do tipo flooding. A infecção começou no segundo 2.643, e os desenvolvedores do conjunto de dados lançaram o ataque DDoS no segundo 5.632. No entanto, ao analisar detalhadamente o conjunto de dados, verificou-se que a primeira ocorrência de atividade associada ao *bot* aparece somente no segundo 5.640. Por esse motivo, as avaliações para este *dataset* foram realizadas entre os segundos 441 e 5.639. O segundo 441 foi escolhido como ponto inicial, pois antes dele não há dados completos disponíveis no conjunto, garantindo assim a inclusão de todo o período relevante para a análise. Em conjuntos futuros é possível notar essa mesma diferenciação nos tempos; o motivo se mantém o mesmo.

4.1.1 Cenário de treinamento 1

A divisão dos dados foi a seguinte: 60% Treinamento, 20% Validação e 20% Teste. A Tabela 2 e a Figura 9 apresentam os resultados desse experimento. Tanto para os valores '0' e '1' o sistema atingiu uma precisão de 100%, um *recall* de 100% e um *f1-score* de 100%. Esses resultados indicam a ocorrência de overfitting durante o treinamento. Para mitigar esse problema, foram realizadas alterações nas porcentagens dos dados utilizados em cada etapa, cujos resultados estão apresentados nos cenários subsequentes.

Classe	**0**	**1**	**Acurácia**	**Precisão**	**Recall**	**F1-score**
0	973	0	100%	1.0	1.0	1.0
1	0	62	100%	1.0	1.0	1.0

Tabela 2 – Matriz de confusão Conjunto 1 - Cenário 1.

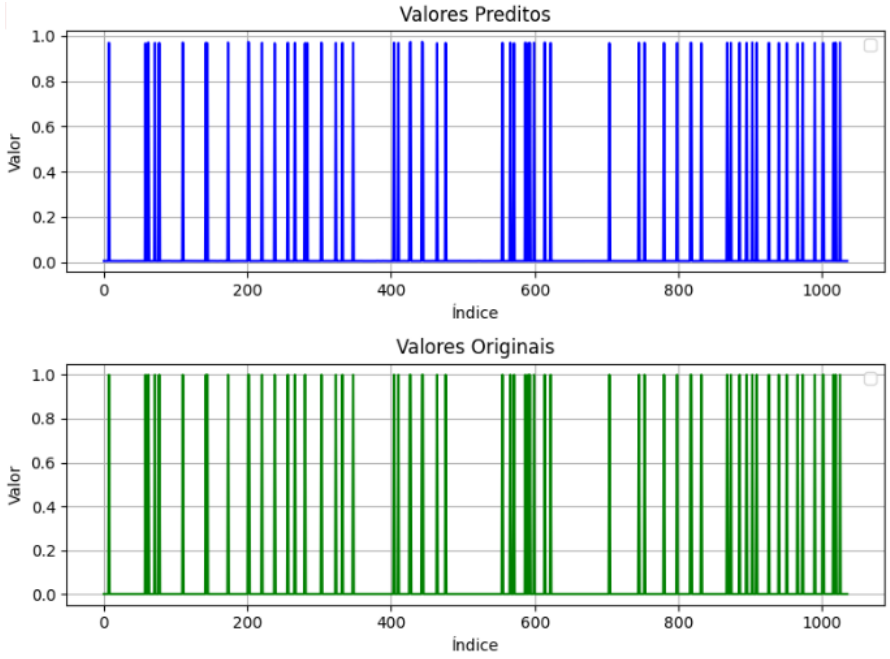


Figura 9 – Gráfico comparando os valores preditos (*logits*) com os valores originais no Conjunto 1 - Cenário 1.

4.1.2 Cenário de treinamento 2

A divisão dos dados foi a seguinte: 40% Treinamento, 10% Validação e 50% Teste. A Tabela 3 e a Figura 10 apresentam os resultados desse experimento. Para os valores '0' o sistema atingiu uma precisão de 99.92%, um *recall* de 99.37% e um *f1-score* de 99.64%, já para os valores '1' o sistema gerou uma precisão de 92.75%, um *recall* de 98.97% e um *f1-score* de 95.76%s.Esses resultados indicam que o modelo tem um desempenho consistente em identificar fluxos normais ('0'), mas apresenta uma ligeira dificuldade em prever corretamente fluxos associados a *bots* ('1'). Isso pode ser atribuído ao desbalanceamento do *dataset* ou à maior complexidade dos padrões associados aos fluxos maliciosos. A alta precisão geral sugere que o modelo está adaptado ao conjunto de dados, mas a diferença entre precisão e *recall* nos valores '1' indica que ajustes no treinamento ou nas métricas de validação poderiam ser explorados para melhorar esse aspecto.

Classe	**0**	**1**	**Acurácia**	**Precisão**	**Recall**	**F1-score**
0	2379	2	99.34%	0.9992	0.9937	0.9964
1	15	192	99.34%	0.9275	0.9897	0.9576

Tabela 3 – Matriz de confusão Conjunto 1 - Cenário 2.

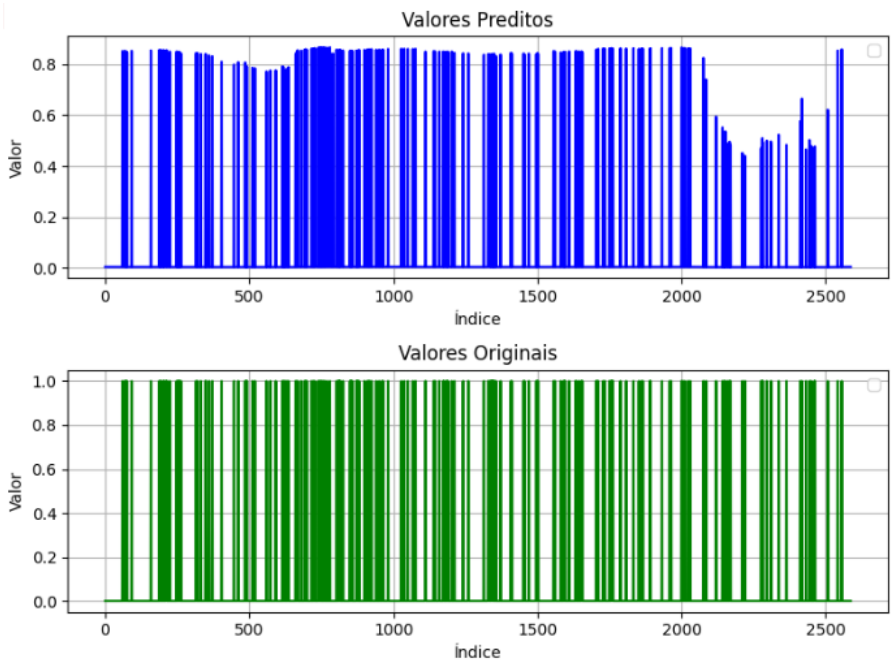


Figura 10 – Gráfico comparando os valores preditos (*logits*) com os valores originais no Conjunto 1 - Cenário 2.

4.1.3 Cenário de treinamento 3

A divisão dos dados foi a seguinte: 30% Treinamento, 10% Validação e 60% Teste. A Tabela 4 e a Figura 11 apresentam os resultados desse experimento. Para os valores '0' o sistema atingiu uma precisão de 91.66%, um *recall* de 100% e um *f1-score* de 8.34%, já para os valores '1' o sistema gerou uma precisão, *recall* e *f1-score* como 0 para todos. Esses resultados revelam que o modelo teve dificuldades significativas em prever fluxos associados a *bots* ('1') nesse cenário. A redução na quantidade de dados de treinamento pode ter comprometido a capacidade do modelo de aprender padrões suficientes para identificar corretamente esses fluxos. Essa situação pode indicar que o modelo depende fortemente de uma quantidade mínima de dados representativos durante o treinamento para ser eficaz.

Classe	**0**	**1**	**Acurácia**	**Precisão**	**Recall**	**F1-score**
0	2847	0	91.66%	1.0	0.0834	0.9565
1	259	0	91.66%	0	0	0

Tabela 4 – Matriz de confusão Conjunto 1 - Cenário 3.

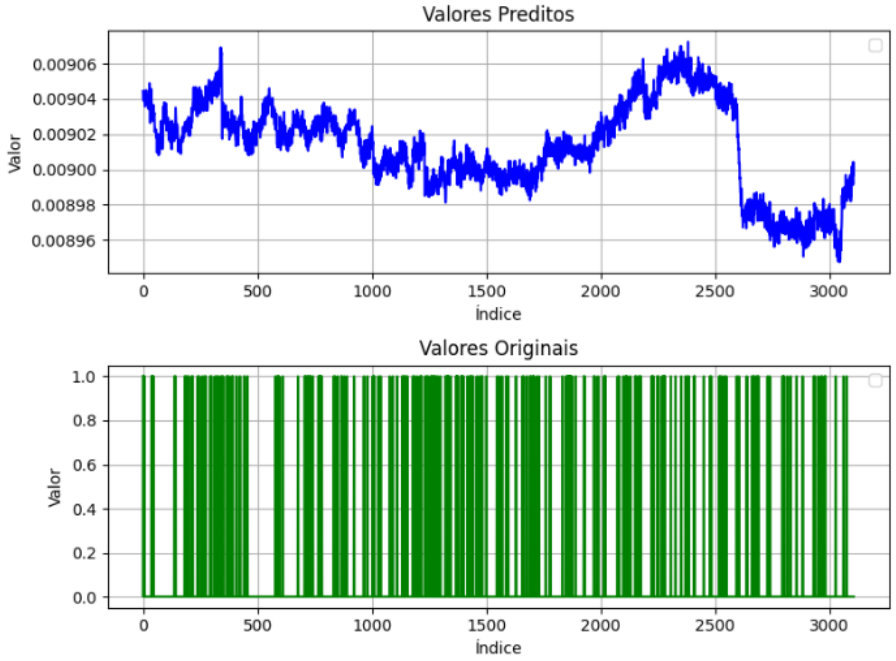


Figura 11 – Gráfico comparando os valores preditos (*logits*) com os valores originais no Conjunto 1 - Cenário 3.

4.2 Experimentos com conjunto de dados 2

O conjunto de dados 2 é referente à captura 52 do conjunto de dados CTU-13 (García et al., 2014). Este *dataset* tem três *bots* que enviam pacotes antes do início do ataque. O

conjunto de dados possui 555 MB, contém 6.336.398 pacotes, dura 972 segundos e inclui um ataque de inundação ICMP. A infecção começou no segundo 527, e os desenvolvedores do conjunto de dados lançaram o ataque DDoS no segundo 778. As avaliações para este experimento foram feitas do segundo 48 até o segundo 801.

4.2.1 Cenário de treinamento 1

A divisão dos dados foi a seguinte: 40% Treinamento, 10% Validação e 50% Teste. A Tabela 5 e a Figura 12 apresentam os resultados desse experimento. Para os valores '0' o sistema atingiu uma precisão de 91.06%, um *recall* de 91.06% e um *f1-score* de 8.34%, já para os valores '1' o sistema gerou uma precisão, *recall* e *f1-score* como 0 para todos. Essa incapacidade de prever corretamente fluxos maliciosos ('1') reforça a ideia de que o modelo pode não estar aprendendo adequadamente padrões relevantes para identificar *bots* nesse conjunto de dados específico. A combinação de fatores, como a divisão de dados ou o possível desbalanceamento, pode estar influenciando negativamente o desempenho do modelo.

Classe	**0**	**1**	**Acurácia**	**Precisão**	**Recall**	**F1-score**
0	275	0	91.06%	1.0	0.9106	0.9532
1	27	0	91.06%	0	0	0

Tabela 5 – Matriz de confusão Conjunto 2 - Cenário 1.

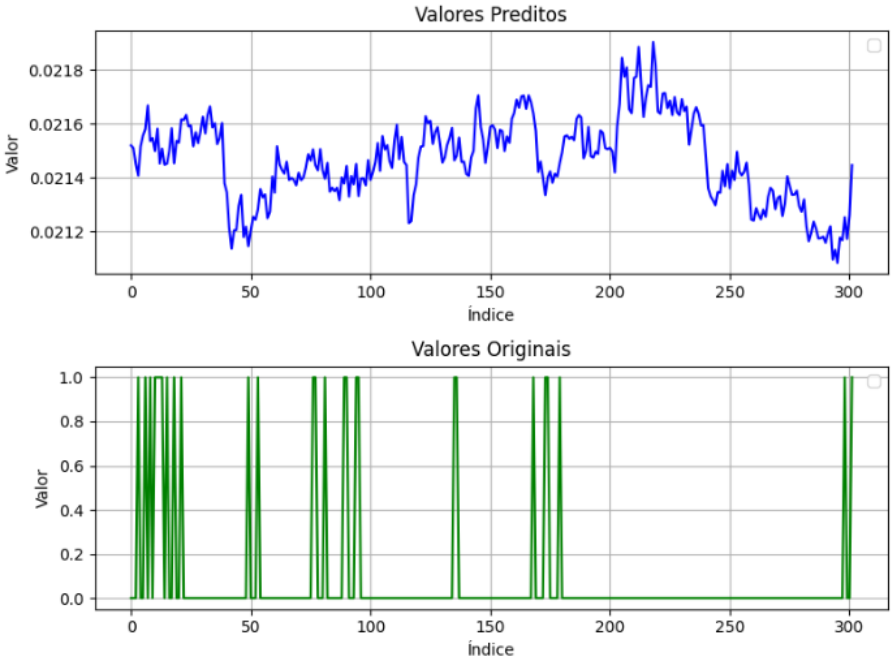


Figura 12 – Gráfico comparando os valores preditos (*logits*) com os valores originais no Conjunto 2 - Cenário 1.

4.2.2 Cenário de treinamento 2

A divisão dos dados foi a seguinte: 60% Treinamento, 5% Validação e 35% Teste. A Tabela 6 e a Figura 13 apresentam os resultados desse experimento. Para os valores '0' o sistema atingiu uma precisão de 100%, um *recall* de 93.58% e um *f1-score* de 96.69%, já para os valores '1' o sistema gerou uma precisão, *recall* e *f1-score* como 0 para todos. Embora o desempenho para fluxos normais ('0') seja excelente, a incapacidade do modelo de prever corretamente fluxos maliciosos ('1') permanece. Isso pode indicar uma limitação no processo de treinamento ou na própria representação dos dados maliciosos no conjunto de dados. Estratégias como aumento de dados ou balanceamento poderiam ser consideradas para mitigar essa falha.

Classe	**0**	**1**	**Acurácia**	**Precisão**	**Recall**	**F1-score**
0	248	0	93.58%	1.0	0.9358	0.9669
1	17	0	93.58%	0	0	0

Tabela 6 – Matriz de confusão Conjunto 2 - Cenário 2.

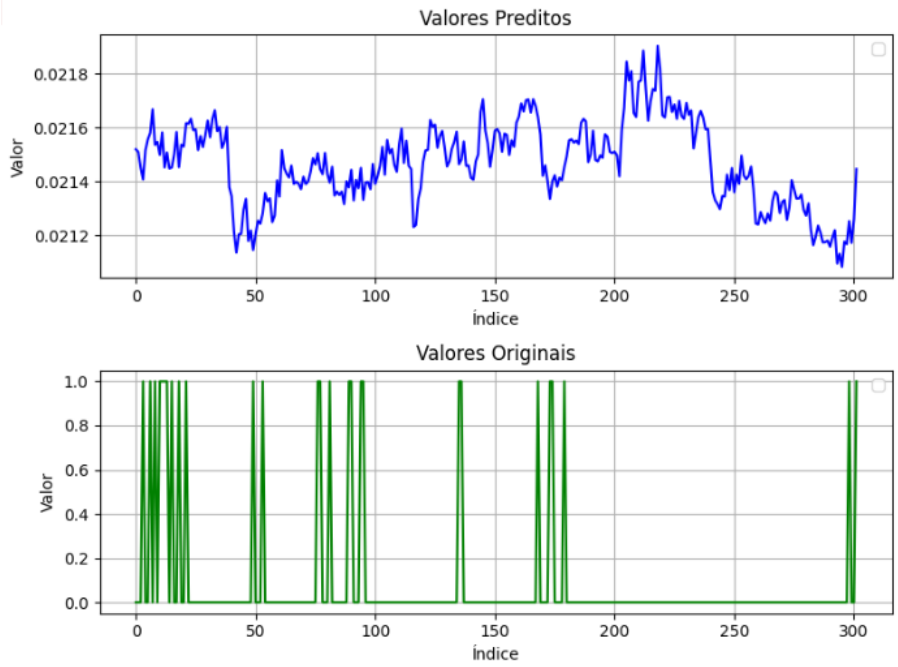


Figura 13 – Gráfico comparando os valores preditos (*logits*) com os valores originais no Conjunto 2 - Cenário 2.

4.2.3 Cenário de treinamento 3

A divisão dos dados foi a seguinte: 80% Treinamento, 5% Validação e 15% Teste. A Tabela 7 e a Figura 14 apresentam os resultados desse experimento. Para os valores '0' o sistema atingiu uma precisão de 100%, um *recall* de 98.25% e um *f1-score* de 99.12%, já para os valores '1' o sistema gerou uma precisão, *recall* e *f1-score* como 0 para todos. A alta quantidade de dados de treinamento melhorou o desempenho do modelo para fluxos normais ('0'), mas não trouxe melhorias para os fluxos maliciosos ('1'). Isso reforça a hipótese de que o modelo está enfrentando dificuldades em aprender padrões representativos para identificar *bots*, mesmo com um aumento significativo de dados de treinamento.

Classe	**0**	**1**	**Acurácia**	**Precisão**	**Recall**	**F1-score**
0	112	0	98.25%	1.0	0.9825	0.9912
1	2	0	98.25%	0	0	0

Tabela 7 – Matriz de confusão Conjunto 2 - Cenário 3.

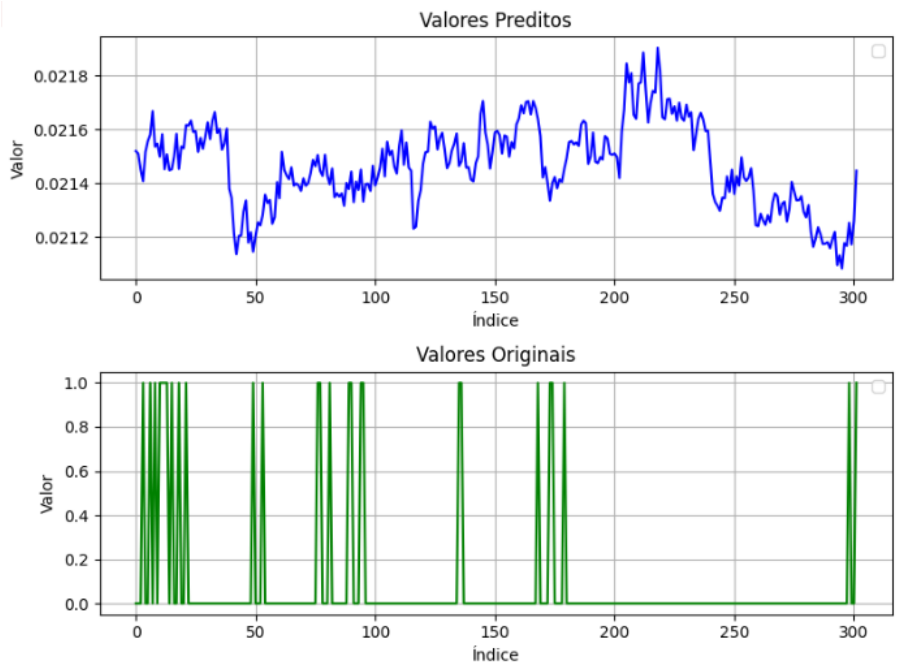


Figura 14 – Gráfico comparando os valores preditos (*logits*) com os valores originais no Conjunto 2 - Cenário 3.

4.2.4 Cenário de treinamento 4

Para este cenário, voltou-se a divisão para 60% Treinamento, 5% Validação e 35% Teste, mas a quantidade de neurônios e de camadas foi duplicada. A Tabela 8 e a Figura 15 apresentam os resultados desse experimento. É possível ver que os resultados foram os mesmo do cenário 2, mesmo com o aumento na complexidade do modelo. Além dessa configuração, foi testada mais algumas, na qual a quantidade de neurônios e camadas triplicou e quadruplicou, mas não ocorreu uma melhora significativa no resultado, com as métricas continuando as mesmas que a deste cenário. Esse resultado indica que o modelo, na configuração original, já era suficiente para capturar os padrões presentes nos dados. O aumento da complexidade não trouxe benefícios adicionais, sugerindo que os limites do modelo já haviam sido alcançados com a configuração anterior.

Classe	**0**	**1**	**Acurácia**	**Precisão**	**Recall**	**F1-score**
0	112	0	98.25%	1.0	0.9825	0.9912
1	2	0	98.25%	0	0	0

Tabela 8 – Matriz de confusão Conjunto 2 - Cenário 4.

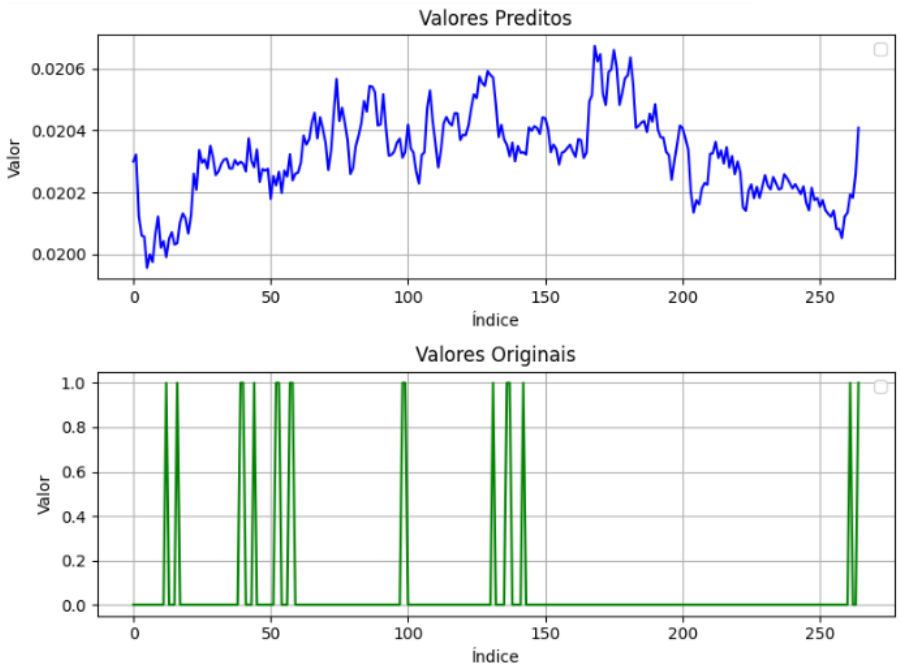


Figura 15 – Gráfico comparando os valores preditos (*logits*) com os valores originais no Conjunto 2 - Cenário 4.

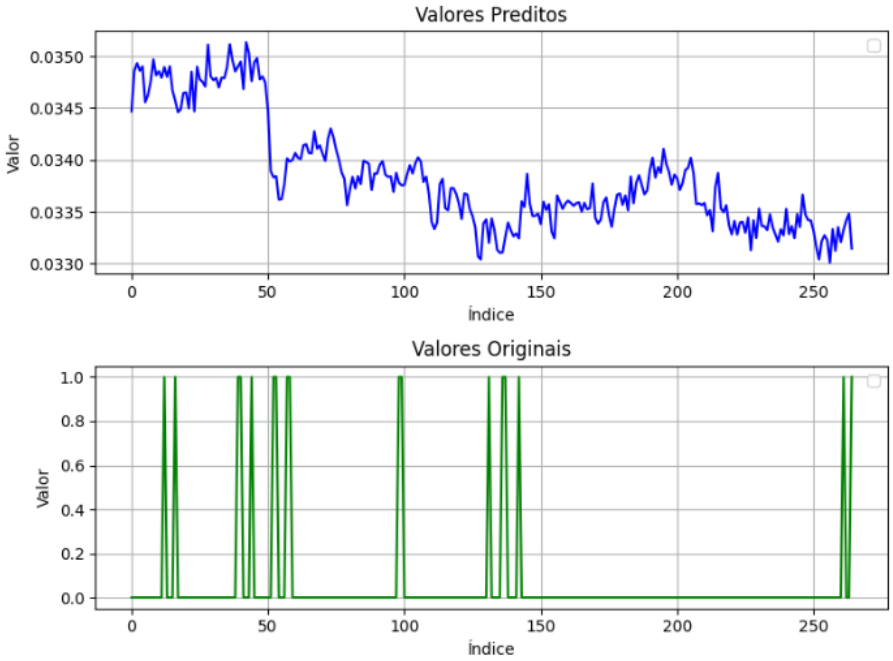


Figura 16 – Gráfico comparando os valores preditos (*logits*) com os valores originais no Conjunto 2 - Cenário 5.

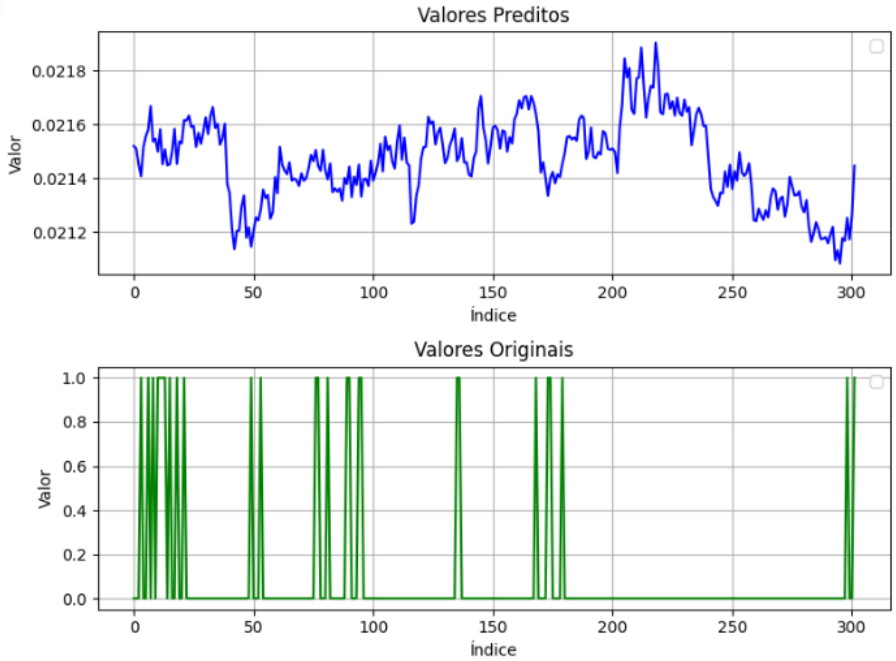


Figura 17 – Gráfico comparando os valores preditos (*logits*) com os valores originais no Conjunto 2 - Cenário 6.

4.3 Experimentos com conjunto de dados 3

Os cenários a seguir foram feitos utilizando como base no *dataset* CICDDoS2019 (Sharafaldin et al., 2019). Este conjunto de dados contém 19 eventos de ataques DDoS no qual a rede da vítima está ligada ao atacante via internet. Foi utilizado o primeiro ataque DDoS no primeiro dia de captura. O tamanho do tráfego de rede capturado foi de 27 GB, contendo 61 milhões de pacotes, registrados em 2024 segundos. Durante a gravação, foi lançado um ataque do tipo PortMap no 1484º segundo. Por fim, as avaliações para este experimento foram feitas do segundo 101 até o segundo 1188. É interessante ressaltar que este foi o *dataset* com a menor discrepância entre o número registrado de fluxo normal '0' para com *bot* '1'.

4.3.0.1 Cenário de treinamento 1

A divisão dos dados foi a seguinte: 30% Treinamento, 5% Validação e 65% Teste. A Tabela 9 e a Figura 18 apresentam os resultados desse experimento. Para os valores '0' o sistema atingiu uma precisão de 91.06%, um *recall* de 91.06% e um *f1-score* de 8.34%, já para os valores '1' o sistema gerou uma precisão, *recall* e *f1-score* como 0 para todos. Esse resultado reflete a dificuldade do modelo em lidar com cenários altamente desbalanceados ou com características mais complexas associadas aos fluxos maliciosos. A discrepância entre o desempenho para fluxos normais e maliciosos sugere a necessidade de técnicas como ajuste de hiperparâmetros ou aumento da representatividade dos dados de *bots* no treinamento.

Classe	**0**	**1**	**Acurácia**	**Precisão**	**Recall**	**F1-score**
0	517	0	61.69%	1.0	0.6169	0.7631
1	321	0	61.69%	0	0	0

Tabela 9 – Matriz de confusão Conjunto 3 - Cenário 1.

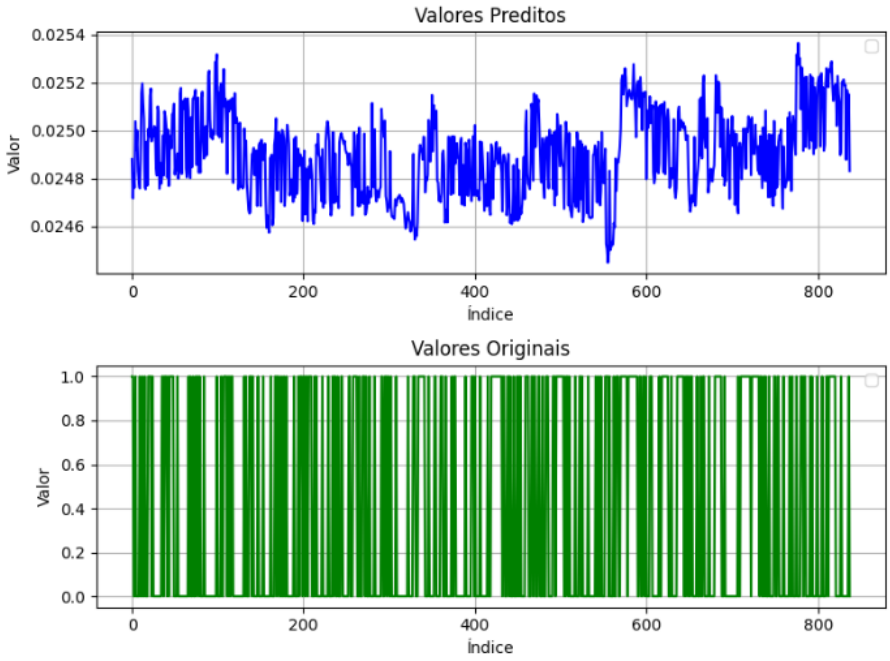


Figura 18 – Gráfico comparando os valores preditos (*logits*) com os valores originais no Conjunto 3 - Cenário 1.

4.3.1 Cenário de treinamento 2

A divisão dos dados foi a seguinte: 40% Treinamento, 5% Validação e 55% Teste. A Tabela 10 e a Figura 19 apresentam os resultados desse experimento. Para os valores '0' o sistema atingiu uma precisão de 100%, um *recall* de 93.67% e um *f1-score* de 96.73%, já para os valores '1' o sistema gerou uma precisão de 95.91%, um *recall* de 89.64% e um *f1-score* de 94.54%s. Neste cenário, o modelo apresentou um desempenho equilibrado entre fluxos normais ('0') e fluxos maliciosos ('1'). O aumento da proporção de dados de treinamento, aliado a um ajuste mais adequado dos dados de teste, parece ter permitido ao modelo aprender melhor os padrões associados a ambos os tipos de fluxo.

Classe	**0**	**1**	**Acurácia**	**Precisão**	**Recall**	**F1-score**
0	429	0	95.91%	1.0	0.9367	0.9673
1	29	251	95.91%	0.8964	1	0.9454

Tabela 10 – Matriz de confusão Conjunto 3 - Cenário 2.

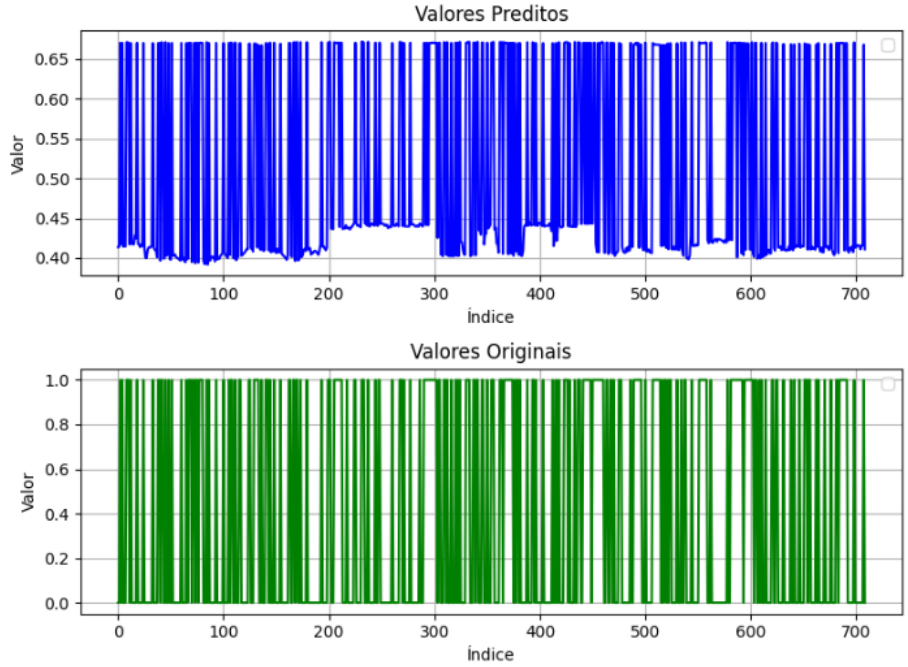


Figura 19 – Gráfico comparando os valores preditos (*logits*) com os valores originais no Conjunto 3 - Cenário 2.

4.4 Experimentos misturando o conjunto de dados 1 e 2

Para a realização dos testes desde conjunto foi adotada uma abordagem diferente dos cenários anteriores, avaliando a capacidade do *FT-Transformer* de generalizar entre capturas distintas do mesmo conjunto de dados. O modelo foi treinado com a captura 51

do CTU-13 (García et al., 2014) e testado com a captura 52 do mesmo conjunto. A divisão dos dados para o treinamento e validação foi de 40% e 10%, respectivamente, enquanto 60% da captura 52 foi utilizado para teste (neste caso referente aos 60% finais do *dataset*).

A Tabela 11 e a Figura 20 apresentam os resultados desse experimento. Para os valores '0' o sistema atingiu uma precisão de 100%, um *recall* de 99.43% e um *f1-score* de 99.72%, já para os valores '1' o sistema gerou uma precisão de 93.10%, um *recall* de 100% e um *f1-score* de 96.43%s. Esses resultados mostram que o modelo foi capaz de generalizar bem entre diferentes capturas do mesmo conjunto de dados, mantendo alta precisão para fluxos normais ('0') e um desempenho sólido para fluxos maliciosos ('1'). Isso sugere que o modelo conseguiu aprender padrões relevantes que se repetem entre as capturas, mesmo diante de possíveis variações nos dados. No entanto, a ligeira queda na precisão para os valores '1' pode ser atribuída a variações específicas entre as capturas, destacando a importância de treinar o modelo com dados mais variados.

Classe	**0**	**1**	**Acurácia**	**Precisão**	**Recall**	**F1-score**
0	351	0	99.47%	1.0	0.9943	0.9972
1	2	27	99.47%	0.9310	1	0.9643

Tabela 11 – Matriz de confusão Conjunto 4

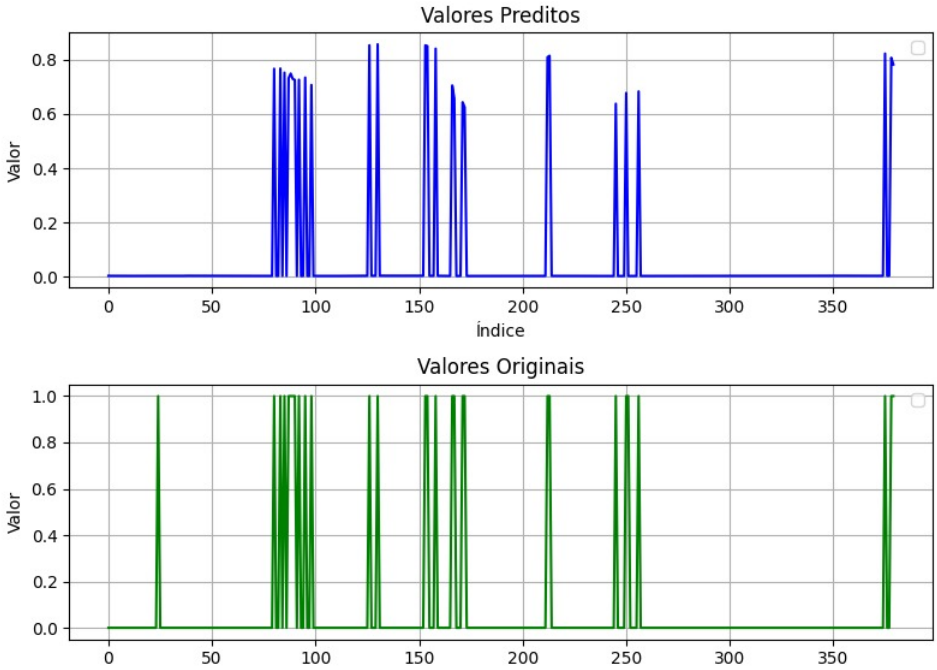


Figura 20 – Gráfico comparando os valores preditos (*logits*) com os valores originais no Conjunto 4.

4.5 Experimentos misturando o conjunto de dados 1 e 3

Em vista dos resultados do experimento anterior, tentou-se averiguar ainda mais os limites do FT-Transformer, foi testada agora a capacidade dele de generalizar entre diferentes *datasets*. O modelo foi treinado com a captura 51 do CTU-13 (García et al., 2014) e testado com o *dataset* CICDDoS2019 (Sharafaldin et al., 2019), um cenário completamente distinto em termos de estrutura e características dos dados. Diante disso, decidiu-se utilizar 40% e 10% dos dados da captura 51 para treinamento e validação respectivamente, já para o *dataset* CICDDoS2019 foi utilizado 50% dos dados para teste(neste caso seria referente aos 50% finais do conjunto).

A Tabela 12 e a Figura 21 apresentam os resultados desse experimento. Para os valores '0' o sistema atingiu uma precisão de 100%, um *recall* de 92.94% e um *f1-score* de 96.34%, já para os valores '1' o sistema gerou uma precisão de 88.97%, um *recall* de 100% e um *f1-score* de 94.16%s. Esses resultados são notáveis, considerando a discrepância significativa entre os dois conjuntos de dados. O *FT-Transformer* conseguiu generalizar de maneira satisfatória, especialmente para fluxos normais ('0'), mantendo precisão.

Ao analisar os valores dos *logits*, observou-se uma melhoria importante na confiança do modelo em relação aos fluxos maliciosos ('1'). No treinamento com *dataset* original, os *logits* máximos para essa classe atingiam valores de aproximadamente 0.7, enquanto neste experimento os valores aumentaram para a faixa de 0.8 a 0.85. Isso indica que o modelo não apenas manteve um bom desempenho geral, mas também demonstrou maior certeza em suas predições para fluxos maliciosos. Estratégias adicionais, como o treinamento com dados mais variados ou o uso de técnicas de aumento de dados, poderiam reforçar ainda mais essa capacidade de generalização, mitigando as limitações identificadas.

Classe	**0**	**1**	**Acurácia**	**Precisão**	**Recall**	**F1-score**
0	382	0	95.50%	1.0	0.9294	0.9634
1	24	234	95.50%	0.8897	1	0.9416

Tabela 12 – Matriz de confusão Conjunto 5

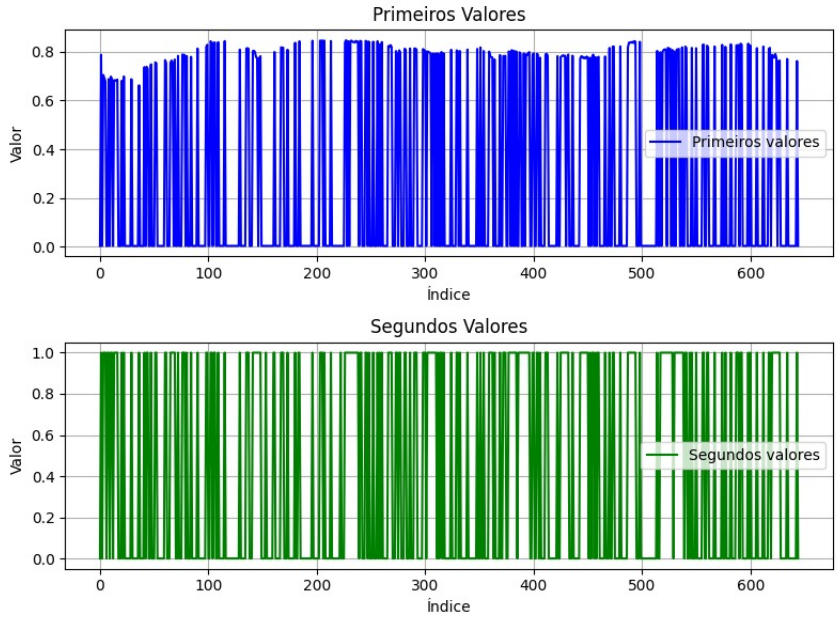


Figura 21 – Gráfico comparando os valores preditos (*logits*) com os valores originais no Conjunto 5.

4.6 Discussão

Os resultados obtidos nos experimentos demonstram a robustez e as limitações do FT-Transformer na predição de ataques DDoS em cenários variados. Como evidenciado nos diferentes testes realizados, o modelo apresentou desempenho sólido em várias configurações, mas também revelou desafios importantes relacionados à generalização e ao balanceamento dos dados. Nos experimentos realizados com a captura 51 do CTU-13, observou-se inicialmente um desempenho perfeito (100% de precisão, *recall* e F1-score) no Cenário 1, com 60% dos dados para treinamento. Esses resultados indicaram a ocorrência de *overfitting*, levando à necessidade de ajustar as divisões dos dados. Nos cenários subsequentes, ao reduzir a proporção de dados para treinamento, o modelo apresentou resultados mais realistas, mas começou a enfrentar dificuldades em identificar fluxos maliciosos ('1'), como no Cenário 3, onde o F1-score para *bots* caiu para 0. Essa limitação sugere que o FT-Transformer depende de uma quantidade mínima de dados representativos durante o treinamento para manter sua eficácia.

Na captura 52 do CTU-13, os resultados seguiram um padrão semelhante, com forte desempenho para fluxos normais ('0') e dificuldades para prever fluxos maliciosos ('1') em configurações com menos dados de treinamento. No entanto, no Cenário 4, mesmo com ajustes na complexidade do modelo, não houve melhorias significativas nos resultados, indicando que o aumento de neurônios e camadas não trouxe benefícios. Isso reforça a ideia de que a qualidade e a representatividade dos dados são mais importantes que a complexidade do modelo em si.

O Experimento 3, utilizando o *dataset* CICDDoS2019, apresentou desafios adicionais devido à menor discrepância entre fluxos normais e maliciosos. Apesar disso, o modelo conseguiu manter um desempenho equilibrado em cenários como o Cenário 2, com F1-scores superiores a 94% para ambos os tipos de fluxo. Esse experimento destacou a capacidade do *FT-Transformer* de lidar com *datasets* mais equilibrados, mas também evidenciou a necessidade de ajustes nas divisões para otimizar o aprendizado do modelo.

Os Experimentos 4 e 5, que avaliaram a capacidade de generalização do *FT-Transformer* entre diferentes conjuntos de dados, trouxeram *insights* importantes. No Experimento 4, o modelo foi capaz de generalizar bem entre capturas distintas do CTU-13, apresentando desempenho sólido tanto para fluxos normais quanto para maliciosos. Já no Experimento 5, a discrepância significativa entre o CTU-13 e o CICDDoS2019 resultou em uma ligeira queda de desempenho, especialmente para fluxos maliciosos ('1'). Apesar disso, a análise dos *logits* revelou um aumento nos valores preditos para fluxos maliciosos, aproximadamente 20% de melhora. Esses valores indicam que o modelo apresentou maior confiança em suas predições para fluxos maliciosos, mesmo diante de diferenças estruturais entre os *datasets*. Esse comportamento destaca uma evolução no aprendizado do modelo, embora desafios em cenários com variações significativas ainda persistam.

De forma geral, os resultados obtidos confirmam o potencial do *FT-Transformer*

para predição de ataques DDoS, mas também destacam áreas que requerem atenção. A dependência do modelo em relação à quantidade e à representatividade dos dados é evidente, sendo necessário um conjunto de dados diversificado e equilibrado para maximizar seu desempenho. Além disso, a análise dos *logits* gerados pelo modelo mostrou-se uma ferramenta valiosa para entender suas decisões e identificar possíveis discrepâncias, o que pode ser explorado em trabalhos futuros para ajustar ainda mais o modelo.

Por fim, considerando o contexto do trabalho, o *FT-Transformer* demonstrou ser uma escolha adequada para a predição de ataques DDoS, apresentando resultados consistentes em diferentes cenários. No entanto, para aplicações práticas, é essencial considerar estratégias como o aumento de dados, balanceamento e validação cruzada para melhorar sua robustez e capacidade de generalização em ambientes reais.

5 Conclusão

Este trabalho deu continuidade à investigação do uso do *FT-Transformer*, um modelo de rede neural baseado em Transformer, que se destaca pela sua capacidade de capturar relações complexas em dados sequenciais, como os fluxos de tráfego de rede. Essa investigação é relevante, pois ataques DDoS são uma ameaça crescente para a segurança de redes e sistemas, resultando em grandes perdas financeiras para as empresas, e a predição eficaz desses ataques pode ajudar na prevenção de danos e na melhoria da segurança cibernética. A pesquisa explorou como o *FT-Transformer* pode ser aplicado para identificar padrões em grandes volumes de dados de tráfego, uma tarefa desafiadora devido à variação nos ataques e ao desbalanceamento das classes nos dados.

Um aspecto importante do *FT-Transformer* é o uso do *token* especial [CLS], que atua como uma representação agregada da sequência de entrada. Nos experimentos realizados, esse *token* mostrou ser fundamental para lidar com o desbalanceamento de classes, pois ele permite que o modelo extraia informações globais relevantes tanto de fluxos normais quanto maliciosos, independentemente da frequência relativa de cada classe nos dados. Essa abordagem auxilia o modelo a priorizar características discriminantes gerais, em vez de apenas refletir o desbalanceamento presente nos dados, contribuindo para métricas mais equilibradas, como o F1-score.

Os testes realizados permitiram responder diretamente à premissa inicial de que o *FT-Transformer* pode ser uma solução eficaz para a predição de ataques DDoS. No experimento que envolveu treinamento com a captura 51 do CTU-13 e teste com o CICDDoS2019, o modelo alcançou uma acurácia de 95%, entre diferentes *datasets*, o que demonstra sua capacidade de generalização em dados de características semelhantes aos de treinamento. Adicionalmente, o modelo apresentou precisão de 99% para fluxos normais em quase todos os experimentos, evidenciando que ele é eficaz em cenários onde as características dos dados são consistentes. No entanto, desafios persistem na detecção de fluxos maliciosos, especialmente em situações de desbalanceamento entre as classes ou com características distintas entre os conjuntos de treinamento e teste.

A análise dos *logits* gerados pelo modelo revelou informações importantes sobre o grau de confiança nas predições, permitindo identificar padrões que ajudaram a ajustar configurações e validar as decisões do modelo. Por exemplo, a análise mostrou que o modelo é capaz de identificar fluxos normais com alta certeza, mas que enfrenta dificuldades em cenários onde os fluxos maliciosos são escassos ou têm características atípicas.

Entre os avanços em relação ao TCC1, destacam-se os resultados positivos em experimentos que exploraram a generalização do modelo. Os testes inter-capturas (Seção 4.4) demonstraram que o *FT-Transformer* mantém um desempenho consistente, com métricas altas para fluxos normais, enquanto os testes inter-*datasets* (Seção 4.5) evidenciaram

que sua eficácia é impactada pela diversidade e representatividade dos dados usados no treinamento.

Por fim, os resultados deste trabalho validam o *FT-Transformer* como uma solução promissora para a predição de ataques DDoS, mas também destacam desafios importantes. Estratégias complementares, como aumento de dados, balanceamento de classes e ajuste de hiperparâmetros, são necessárias para lidar com cenários mais complexos. Com esses avanços, espera-se que o modelo possa ser ainda mais eficaz em aplicações reais, contribuindo para a segurança cibernética e a mitigação dos impactos de ataques DDoS.

5.1 Recomendações Futuras

Com base nos resultados e limitações identificados, recomenda-se que trabalhos futuros explorem estratégias de aumento de dados, como a sintetização de amostras maliciosas, bem como métodos de balanceamento para melhorar o aprendizado de padrões relacionados a fluxos maliciosos. Também seria valioso investigar variações arquiteturais, como combinações do *FT-Transformer* com modelos baseados em Redes Neurais Convolucionais (CNNs) ou Redes Recorrentes (RNNs), para explorar soluções mais robustas. Além disso, incorporar métricas de importância de atributos poderia proporcionar uma compreensão mais profunda sobre as características que influenciam a detecção de ataques.

Outro ponto relevante é a avaliação do modelo em ambientes reais de monitoramento de rede, analisando aspectos como latência e capacidade de resposta em tempo real. Estudos mais aprofundados também poderiam focar na capacidade do modelo de generalizar entre *datasets* distintos, como nos experimentos finais deste trabalho, buscando compreender melhor as limitações e as possibilidades de ajuste entre dados com diferentes padrões estruturais. Além disso, seria importante realizar uma separação mais precisa entre *bots* "normais"(como scrappers) e aqueles efetivamente utilizados em ataques DDoS. Essa distinção pode melhorar a acurácia do FT-Transformer, permitindo um aprendizado mais refinado e relevante para a predição de ameaças reais.

Por fim, (Gorishniy et al., 2023) mencionaram que não investiram muito tempo para melhorar os valores dos hiperparâmetros, logo a realização de um estudo mais aprofundado sobre a sensibilidade dos hiperparâmetros pode ajudar a identificar quais ajustes proporcionam maior impacto no desempenho, facilitando futuras otimizações. Essas iniciativas poderiam contribuir significativamente para avançar o estado da arte na predição de ataques DDoS, consolidando o FT-Transformer como uma ferramenta eficaz para a segurança cibernética.

Referências

ABDLHAMED, Mohamed et al. Intrusion Prediction Systems. In: *Information Fusion for Cyber-Security Analytics*. Edição: Izzat M Alsmadi, George Karabatis e Ahmed Aleroud. Cham: Springer International Publishing, 2017. P. 155–174. ISBN 978-3-319-44257-0. Citado 1 vez na página 18.

ADEGBOYEGA, Abiola. An Adaptive Score Model for Effective Bandwidth Prediction and Provisioning in the Cloud Network. In: 2015 IEEE Globecom Workshops (GC Wkshps). 2015. P. 1–7. DOI: 10.1109/GLOCOMW.2015.7414021. Citado 1 vez na página 18.

BEBIS, G.; GEORGIOPOULOS, M. Feed-forward neural networks. *IEEE Potentials*, v. 13, n. 4, p. 27–31, 1994. DOI: 10.1109/45.329294. Citado 1 vez na página 23.

BREMLER-BARR, Anat; BROSH, Eli; SIDES, Mor. DDoS attack on cloud auto-scaling mechanisms. In: IEEE INFOCOM 2017 - IEEE Conference on Computer Communications. 2017. P. 1–9. DOI: 10.1109/INFOCOM.2017.8057010. Citado 1 vez na página 17.

CHILD, Rewon et al. *Generating Long Sequences with Sparse Transformers*. 2019. arXiv: 1904.10509 [cs.LG]. Disponível em: <https://arxiv.org/abs/1904.10509>. Citado 1 vez na página 30.

CISCO. *Cisco Annual Internet Report (2018–2023)*. 2023. Acesso em: 01 jun. 2024. Citado 1 vez na página 10.

DEHKORDI, A. Banitalebi; SOLTANAGHAEI, M.; BOROUJENI, F.Z. The DDoS attacks detection through machine learning and statistical methods in SDN. *J Supercomput*, v. 77, p. 2383–2415, 2021. DOI: https://doi.org/10.1007/s11227-020-03323-w. Citado 2 vezes nas páginas 10, 17.

DOULIGERIS, Christos; MITROKOTSA, Aikaterini. DDoS attacks and defense mechanisms: classification and state-of-the-art. *Computer Networks*, v. 44, n. 5, p. 643–666, 2004. ISSN 1389-1286. DOI: https://doi.org/10.1016/j.comnet.2003.10.003. Disponível em: <https://www.sciencedirect.com/science/article/pii/S1389128603004250>. Citado 1 vez na página 16.

FENG, Yaokai et al. Feature Selection for Machine Learning-Based Early Detection of Distributed Cyber Attacks. In: 2018 IEEE 16th Intl Conf on Dependable, Autonomic and Secure Computing, 16th Intl Conf on Pervasive Intelligence and Computing, 4th Intl Conf on Big Data Intelligence and Computing and Cyber Science and Technology Congress(DASC/PiCom/DataCom/CyberSciTech). 2018. P. 173–180. DOI: 10.1109/DASC/PiCom/DataCom/CyberSciTec.2018.00040. Citado 1 vez na página 36.

GARCÍA, S. et al. An empirical comparison of botnet detection methods. *Computers & Security*, v. 45, p. 100–123, 2014. ISSN 0167-4048. DOI: https://doi.org/10.1016/j.cose.2014.05.011. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0167404814000923>. Citado 6 vezes nas páginas 11, 13, 37, 40, 49, 50.

GARCÍA-TEODORO, P. et al. Anomaly-based network intrusion detection: Techniques, systems and challenges. *Computers & Security*, v. 28, n. 1, p. 18–28, 2009. ISSN 0167-4048. DOI: https://doi.org/10.1016/j.cose.2008.08.003. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0167404808000692>. Citado 1 vez na página 17.

GIANNOTTI, Livia. *The biggest cyberattacks of 2024*. 2024. Disponível em: https://techmonitor.ai/technology/cybersecurity/biggest-cyberattacks-of-2024. Acesso em: 17 jul. 2024. Citado 1 vez na página 10.

GORISHNIY, Yury et al. *Revisiting Deep Learning Models for Tabular Data*. 2023. arXiv: 2106.11959 [cs.LG]. Disponível em: <https://arxiv.org/abs/2106.11959>. Citado 10 vezes nas páginas 11, 21, 23, 25–30, 32, 33, 35, 55.

H. SALEMI H. ROSTAMI, S. Talatian-Azad. LEAESN: Predicting DDoS attack in healthcare systems based on Lyapunov Exponent Analysis and Echo State Neural Networks. *Multimed Tools Appl*, 2022. ISSN 41455–41476. DOI: https://doi.org/10.1007/s11042-020-10179-y. Disponível em: <https://link.springer.com/article/10.1007/s11042-020-10179-y>. Citado 1 vez na página 18.

HILL, Michael. *DDoS Attack-for-Hire Services Thriving on Dark Web and Cyber Criminal Forums*. Online news article. 4 dez. 2023. Disponível em: <https://www.cshub.com/attacks/news/ddos-attack-for-hire-services-thriving-on-dark-web-and-cyber-criminal-forums>. Citado 1 vez na página 15.

HUANG, Xin et al. *TabTransformer: Tabular Data Modeling Using Contextual Embeddings*. 2020. arXiv: 2012.06678 [cs.LG]. Disponível em: <https://arxiv.org/abs/2012.06678>. Citado 2 vezes nas páginas 10, 29, 30.

JAIN, A.K.; SHUKLA, H.; GOEL, D. A comprehensive survey on DDoS detection, mitigation, and defense strategies in software-defined networks. *Cluster Comput*, 2024. DOI: https://doi.org/10.1007/s10586-024-04596-z. Citado 1 vez na página 10.

JYOTI, Navjot; BEHAL, Sunny. A Meta-evaluation of Machine Learning Techniques for Detection of DDoS Attacks. In: 2021 8th International Conference on Computing for Sustainable Global Development (INDIACom). 2021. P. 522–526. Citado 1 vez na página 36.

KIM, Geunho et al. Predicting heart failure prognosis using deep learning based on FT-transformer. In: 2023 Fourteenth International Conference on Ubiquitous and Future Networks (ICUFN). 2023. P. 932–936. DOI: 10.1109/ICUFN57995.2023.10200998. Citado 1 vez na página 31.

LIMA, Matheus et al. Predição Não-supervisionada de Ataques DDoS por Sinais Precoces e One-Class SVM. In: ANAIS do XXIII Simpósio Brasileiro de Segurança da Informação e de Sistemas Computacionais. Juiz de Fora/MG: SBC, 2023. P. 403–416. DOI: 10.5753/sbseg.2023.233512. Disponível em: <https://sol.sbc.org.br/index.php/sbseg/article/view/27222>. Citado 1 vez na página 12.

LOPEZ, Maribel; BUSCETTO, Mena. *Cybercriminal Attacks Accelerate Global Cybersecurity Crisis According to 1H2021 NETSCOUT Threat Intelligence Report*. 2021. Disponível em: https://www.netscout.com/press-releases/cybercriminal-attacks-accelerate-global-cybersecurity. Acesso em: 01 jun. 2024. Citado 1 vez na página 10.

MATHEWS, Lee. *Angry Gamer Blamed For Most Devastating DDoS Of 2016*. forbes, 2016. Disponível em: https://www.forbes.com/sites/leemathews/2016/11/17/angry-gamer-blamed-for-most-devastating-ddos-of-2016/7052ed4039e4. Acesso em: 24 jul. 2024. Citado 1 vez na página 15.

MIRKOVIC, J.; PRIER, G.; REIHER, P. Attacking DDoS at the source. In: 10TH IEEE International Conference on Network Protocols, 2002. Proceedings. 2002. P. 312–321. DOI: 10.1109/ICNP.2002.1181418. Citado 1 vez na página 17.

MIRKOVIC, Jelena; REIHER, Peter. A taxonomy of DDoS attack and DDoS defense mechanisms. *SIGCOMM Comput. Commun. Rev.*, Association for Computing Machinery, New York, NY, USA, v. 34, n. 2, p. 39–53, abr. 2004. ISSN 0146-4833. DOI: 10.1145/997150.997156. Disponível em: <https://doi.org/10.1145/997150.997156>. Citado 4 vezes nas páginas 15, 18.

NEIRA, Anderson B. de; BORGES, Ligia F. et al. Unsupervised AutoML and Dimensionality Reduction for Autonomous DDoS Attack Prediction, 2023. Emails: abneira@inf.ufpr.br, {ligiaborges, michele}@dcc.ufmg.br, batista@ime.usp.br. Citado 6 vezes nas páginas 32–36.

NEIRA, Anderson Bergamini de; KANTARCI, Burak; NOGUEIRA, Michele. Distributed denial of service attack prediction: Challenges, open issues and opportunities. *Computer Networks*, v. 222, p. 109553, 2023. ISSN 1389-1286. DOI: https://doi.org/10.1016/j.comnet.2022.109553. Disponível em: <https://www.sciencedirect.com/science/article/pii/S1389128622005874>. Citado 2 vezes nas páginas 18, 32.

PELOFSKE, Elijah; URIAS, Vincent; LIEBROCK, Lorie M. *Automated Multi-Language to English Machine Translation Using Generative Pre-Trained Transformers*. 2024. arXiv: 2404.14680 [cs.CL]. Disponível em: <https://arxiv.org/abs/2404.14680>. Citado 1 vez na página 24.

PRAKASH, A. et al. Detection and Mitigation of Denial of Service Attacks Using Stratified Architecture. *Procedia Computer Science*, v. 87, p. 275–280, 2016. ISSN 1877-0509. DOI: 10.1016/j.procs.2016.05.161. Disponível em: <https://www.sciencedirect.com/science/article/pii/S1877050916305014>. Citado 1 vez na página 16.

PRAVEEN, S. Phani et al. Enhanced Intrusion Detection Using Stacked FT-Transformer Architecture. *Journal of Cybersecurity and Information Management (JCIM)*, v. 13, p. 19–29, 2024. ISSN 0167-4048. DOI: 10.54216/JCIM.130202. Disponível em: <https://research.ebsco.com/linkprocessor/plink?id=2c07d0bc-d224-327a-96e3-bf6d48f659eb>. Citado 1 vez na página 30.

RANJAN, Supranamaya et al. DDoS-Shield: DDoS-Resilient Scheduling to Counter Application Layer Attacks. *IEEE/ACM Transactions on Networking*, v. 17, n. 1, p. 26–39, 2009. DOI: 10.1109/TNET.2008.926503. Citado 1 vez na página 17.

RAWSKI, M. Network Topology Mutation as Moving Target Defense for Corporate Networks. *International Journal of Electronics and Telecommunications*, v. 65, p. 571–577, 2019. Nenhuma citação no texto.

RUMELHART, David E; HINTON, Geoffrey E; WILLIAMS, Ronald J. Learning representations by back-propagating errors. *Nature*, Springer, v. 323, n. 6088, p. 533–536, 1986. Nenhuma citação no texto.

SHARAFALDIN, Iman et al. Developing Realistic Distributed Denial of Service (DDoS) Attack Dataset and Taxonomy. In: 2019 International Carnahan Conference on Security Technology (ICCST). 2019. P. 1–8. DOI: 10.1109/CCST.2019.8888419. Citado 4 vezes nas páginas 11, 13, 46, 50.

SILVA, F. S. D. et al. A Taxonomy of DDoS Attack Mitigation Approaches Featured by SDN Technologies in IoT Scenarios. *Sensors (Basel)*, MDPI, v. 20, n. 11, p. 3078, 2020. Published 2020 May 29. DOI: 10.3390/s20113078. Nenhuma citação no texto.

SINGH, Chandrapal; JAIN, Ankit Kumar. A comprehensive survey on DDoS attacks detection & mitigation in SDN-IoT network. *e-Prime - Advances in Electrical Engineering, Electronics and Energy*, v. 8, p. 100543, 2024. ISSN 2772-6711. DOI: https://doi.org/10.1016/j.prime.2024.100543. Disponível em: <https://www.sciencedirect.com/science/article/pii/S2772671124001256>. Citado 1 vez na página 17.

EL-SOFANY, Hosam F.; EL-SEOUD, Samir A.; TAJ-EDDIN, Islam A. T. F. A Case Study of the Impact of Denial of Service Attacks in Cloud Applications. *Journal of Communications*, v. 14, n. 2, p. 153–158, 2019. DOI: 10.12720/jcm.14.2.153-158. Disponível em: <https://www.jocm.us/uploadfile/2019/0121/20190121055600859.pdf>. Citado 3 vezes nas páginas 15, 16.

SONI, Sachin. Masked Multi-Head Attention in Transformer. *Medium*, 2023. Acessado em 24 jan. 2025. Disponível em: <https://medium.com/@sachinsoni600517/masked-multi-head-attention-in-transformer-f3e096d56961>. Citado 2 vezes nas páginas 25, 26.

STRATOSPHERE. *Stratosphere Laboratory Datasets*. 2015. Retrieved March 13, 2020, from https://www.stratosphereips.org/datasets-overview. Citado 1 vez na página 27.

TREMANTE, Michael; ZEJNILOVIC, Sabina; NEWCOMB, Catherine. *Application Security report: 2024 update*. 2024. Disponível em: `https://blog.cloudflare.com/application-security-report-2024-update`. Acesso em: 17 jul. 2024. Citado 2 vezes nas páginas 10, 12.

VASWANI, Ashish et al. *Attention Is All You Need*. 2017. arXiv: `1706.03762 [cs.CL]`. Disponível em: <`https://arxiv.org/abs/1706.03762`>. Citado 11 vezes nas páginas 10, 18, 19, 21, 24–26.

VYAS, Tirth Kiranbhai. *Deep Learning with Tabular Data: A Self-supervised Approach*. 2024. arXiv: `2401.15238 [cs.LG]`. Disponível em: <`https://arxiv.org/abs/2401.15238`>. Citado 1 vez na página 10.

WODECKI, Natalia. *Zscaler ThreatLabz 2023 Enterprise IoT and OT Threat Report Uncovers Manufacturing and Education Sectors Targeted the Most, with Education Realizing a Nearly 1000% Increase in IoT Malware Attacks*. 2023. Disponível em: `https://www.zscaler.com/press/zscaler-threatlabz-finds-400-increase-iot-and-ot-malware-attacks-year-over-year-underscoring`. Acesso em: 26 jul. 2024. Nenhuma citação no texto.

YOGESH PATIL, Rachana; RAGHA, Lata. A rate limiting mechanism for defending against flooding based distributed denial of service attack. In: 2011 World Congress on Information and Communication Technologies. 2011. P. 182–186. DOI: `10.1109/WICT.2011.6141240`. Citado 1 vez na página 17.

ZARGAR, Saman Taghavi; JOSHI, James; TIPPER, David. A Survey of Defense Mechanisms Against Distributed Denial of Service (DDoS) Flooding Attacks. *IEEE Communications Surveys & Tutorials*, v. 15, n. 4, p. 2046–2069, 2013. DOI: `10.1109/SURV.2013.031413.00127`. Citado 1 vez na página 16.

ZHANG, Zhaozhe; ZHANG, Hongpo. An Online Transfer Learning Model for Intrusion Detection using FT-Transformer and KSWIN-Driven Concept Drift Detection Mechanism. In: 2024 5th International Conference on Computer Engineering and Application (ICCEA). 2024. P. 128–131. DOI: `10.1109/ICCEA62105.2024.10603489`. Citado 1 vez na página 31.